

```
SELECT c.CustomerName, SUM(od.Quantity * p.Price)
FROM Customers c JOIN Orders o ...
GROUP BY c.CustomerName HAVING ...;
```

HANDS-ON WORKBOOK

Learn SQL by Solving Problems

Every command from the W3Schools SQL tutorial - with a real database, a problem statement, the SQL solution and the actual result.

246

worked problems

8

linked tables

5

chapters + appendices

MySQL

8 / MariaDB tested

Practice database: ShopDB - an Indian FMCG distributor with customers in 5 countries.

Topic order follows <https://www.w3schools.com/sql/>

How to use this workbook

- 1. Install a database.** Any of these works: MySQL 8 Community Server + MySQL Workbench, XAMPP (includes MariaDB + phpMyAdmin), or an online editor such as the W3Schools SQL editor for SELECT-only practice.
- 2. Build ShopDB.** Run the complete setup script from Chapter 1 (also supplied as *ShopDB_setup.sql*). It is re-runnable: it drops and recreates the database every time, so you can always reset.
- 3. Work in order.** Every topic from the W3Schools SQL menu has a short explanation, the general syntax, then numbered problems. Read the problem, **write your own query first**, run it, and only then compare with the solution and result shown.
- 4. Order matters.** Some problems change data (INSERT, UPDATE, DELETE, ALTER, CREATE). Later results assume you ran earlier problems in sequence. If you get lost, re-run the setup script and replay from the start of the chapter.
- 5. Finish with Chapter 5.** Fifteen business-style challenges that combine joins, grouping, subqueries and functions - the kind of questions asked in interviews.

About the results. Every RESULT box in this book is the real output captured by executing the SQL on MariaDB 10.11 (MySQL-compatible) in exactly this order. On MySQL 8 you get the same rows; only error-message wording can differ slightly. Results for NOW(), RAND(), VERSION() and CURRENT_DATE depend on when and where you run them. Red text marks intentional errors that demonstrate constraints doing their job.

Dialect notes. The solutions use MySQL syntax. Where SQL Server, Oracle or MS Access differ (TOP vs LIMIT, ISNULL vs IFNULL, SELECT INTO, FULL JOIN, EXEC, IDENTITY ...) the difference is shown in a Tip box or in the syntax block, and Chapter 4 ends with a translation table.

The ShopDB practice database

ShopDB models a small Indian food distributor. 8 tables are linked through primary keys (PK) and foreign keys (FK); the teal lines show how they join.

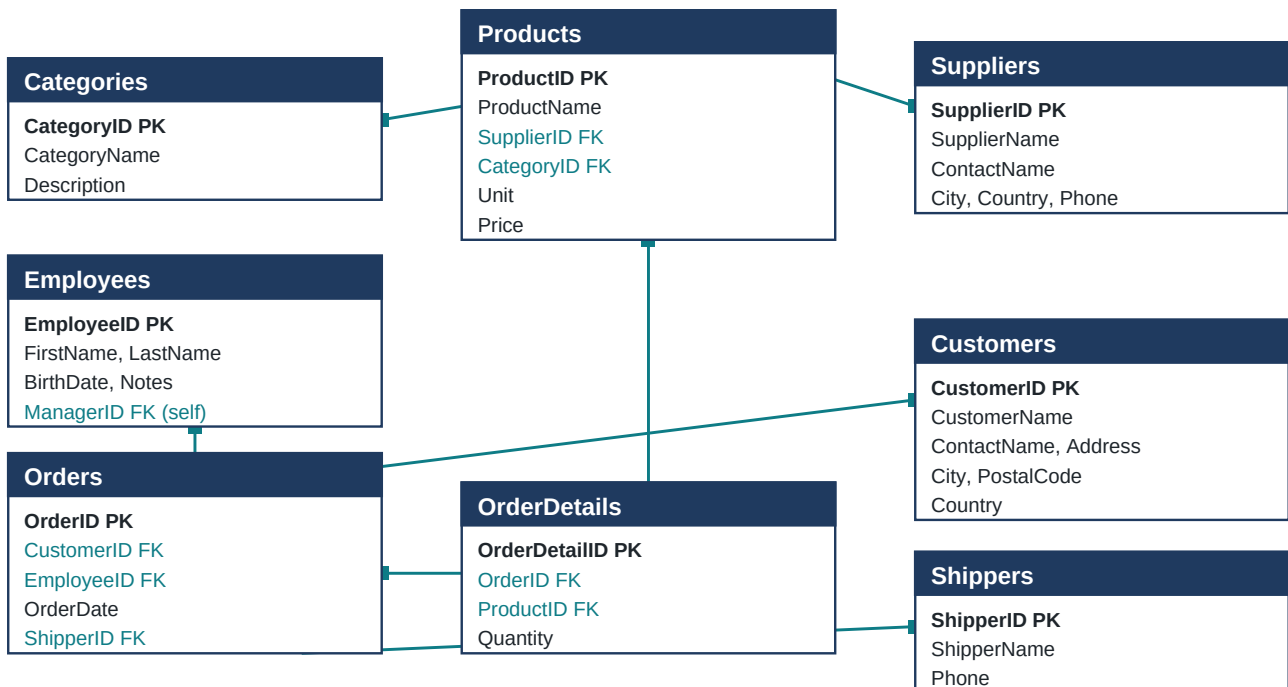


Table	Rows	What it holds / useful quirks for practice
Categories	6	Product groups: Beverages, Snacks, Dairy, Grains, Spices, Sweets
Suppliers	6	Supplier companies in India, UK, Germany, UAE. Kerala Coconut Co has no products (outer joins).
Products	13	Items with price and unit. Gulab Jamun Mix has NULL SupplierID and is never ordered.
Customers	14	Buyers in 5 countries. UAE customers have NULL PostalCode; 2 customers never ordered.
Employees	6	Sales team with ManagerID -> EmployeeID hierarchy (self join). Neha has no orders.
Shippers	3	BlueDart Express, Delhivery, DHL Global
Orders	15	Orders 1001-1015, Jan-Jun 2026
OrderDetails	36	Line items: order + product + quantity (revenue = Quantity x Price)

Contents

1. Build the Practice Database	6
1.1 The complete setup script, step by step	6
1.2 Look at every table	9
2. SQL Tutorial - Querying & Changing Data	14
2.1 SQL Syntax & SELECT	14
2.2 SELECT DISTINCT	15
2.3 WHERE	15
2.4 ORDER BY	17
2.5 AND	19
2.6 OR	20
2.7 NOT	21
2.8 INSERT INTO	22
2.9 NULL Values	23
2.10 UPDATE	25
2.11 DELETE	25
2.12 SELECT TOP / LIMIT / FETCH FIRST	26
2.13 Aggregate Functions	28
2.14 MIN() and MAX()	28
2.15 COUNT()	29
2.16 SUM()	30
2.17 AVG()	31
2.18 LIKE	32
2.19 Wildcards	33
2.20 IN	34
2.21 BETWEEN	35
2.22 Aliases (AS)	37
2.23 JOINS - overview	38
2.24 INNER JOIN	39
2.25 LEFT JOIN	40
2.26 RIGHT JOIN	41
2.27 FULL OUTER JOIN	42
2.28 SELF JOIN	43
2.29 UNION	44
2.30 UNION ALL	45
2.31 GROUP BY	46
2.32 HAVING	47
2.33 EXISTS	48
2.34 ANY	49
2.35 ALL	49
2.36 SELECT INTO	50

2.37 INSERT INTO SELECT	52
2.38 CASE	53
2.39 NULL Functions - IFNULL(), COALESCE(), NULLIF()	54
2.40 Stored Procedures	56
2.41 Comments	58
2.42 Operators	59
3. SQL Database - Structure, Constraints & Security	61
3.1 CREATE DATABASE	61
3.2 DROP DATABASE	61
3.3 BACKUP DATABASE	62
3.4 CREATE TABLE	62
3.5 DROP TABLE and TRUNCATE TABLE	64
3.6 ALTER TABLE	64
3.7 Constraints - overview	67
3.8 NOT NULL	67
3.9 UNIQUE	68
3.10 PRIMARY KEY	69
3.11 FOREIGN KEY	71
3.12 CHECK	72
3.13 DEFAULT	73
3.14 CREATE INDEX	74
3.15 AUTO_INCREMENT	76
3.16 Dates	77
3.17 VIEWS	78
3.18 SQL Injection	80
3.19 SQL Parameters and Prepared Statements	81
3.20 SQL Data Types (MySQL reference)	83
4. SQL Functions - Hands-On	85
4.1 String Functions	85
4.2 Numeric Functions	88
4.3 Date Functions	89
4.4 Advanced Functions	91
4.5 Same task, different database - translation table	93
5. Practice Challenges	94
5.1 The challenge list (try these first!)	94
5.2 Solutions	95
A. Appendix - SQL Keyword Index & Quick Reference	101
A.1 Keyword index	101
A.2 One-page quick reference	102

Build the Practice Database

CREATE DATABASE, CREATE TABLE and INSERT INTO - run this once before anything else

1.1 The complete setup script, step by step

Copy each block into your SQL client and execute it in order (or run the whole ShopDB_setup.sql file at once). Tables are created parents-first so that every FOREIGN KEY points to a table that already exists.

Step 1 - Create and select the database Create a fresh database called ShopDB and make it the active database. DROP ... IF EXISTS lets you re-run the whole script any time to reset everything.

```
DROP DATABASE IF EXISTS ShopDB;
CREATE DATABASE ShopDB;
USE ShopDB;
```

Step 2 - Categories table Product categories (Beverages, Snacks, ...).

```
CREATE TABLE Categories (
  CategoryID      INT AUTO_INCREMENT PRIMARY KEY,
  CategoryName    VARCHAR(50) NOT NULL,
  Description     VARCHAR(255)
);

INSERT INTO Categories (CategoryName, Description) VALUES
('Beverages', 'Tea, coffee and juices'),
('Snacks',    'Namkeen, chips and dry snacks'),
('Dairy',     'Milk products'),
('Grains',    'Rice, flour and cereals'),
('Spices',    'Whole and ground spices'),
('Sweets',    'Indian mithai and dessert mixes');
```

Step 3 - Suppliers table Companies that supply our products. Kerala Coconut Co has no products yet (useful for JOIN examples).

```
CREATE TABLE Suppliers (
  SupplierID      INT AUTO_INCREMENT PRIMARY KEY,
  SupplierName    VARCHAR(100) NOT NULL,
  ContactName     VARCHAR(100),
  City            VARCHAR(50),
  Country         VARCHAR(50),
  Phone           VARCHAR(20)
);

INSERT INTO Suppliers (SupplierName, ContactName, City, Country, Phone) VALUES
('Amul Traders',    'Mahesh Shah',    'Anand',    'India',    '+91-2692-000001'),
('Himalaya Foods',  'Deepa Rawat',    'Dehradun', 'India',    '+91-135-0000002'),
('Spice Route Ltd', 'Oliver Smith',   'London',   'UK',       '+44-20-00000003'),
('Bavaria Imports', 'Klaus Weber',    'Munich',   'Germany',  '+49-89-00000004'),
('Gulf Fresh LLC',  'Ahmed Saeed',    'Dubai',    'UAE',      '+971-4-00000005'),
('Kerala Coconut Co', 'Thomas Mathew', 'Kochi',    'India',    '+91-484-00000006');
```

Step 4 - Products table Each product belongs to one supplier and one category (FOREIGN KEYS). 'Gulab Jamun Mix' has no supplier (NULL).

```
CREATE TABLE Products (  
    ProductID      INT AUTO_INCREMENT PRIMARY KEY,  
    ProductName    VARCHAR(100) NOT NULL,  
    SupplierID     INT NULL,  
    CategoryID     INT,  
    Unit           VARCHAR(50),  
    Price          DECIMAL(10,2) DEFAULT 0,  
    FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID),  
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)  
);  
  
INSERT INTO Products (ProductName, SupplierID, CategoryID, Unit, Price) VALUES  
( 'Masala Chai',      2,      1, '500 g pack', 250.00),  
( 'Filter Coffee',   2,      1, '250 g pack', 320.00),  
( 'Mango Juice',     5,      1, '1 L bottle', 120.00),  
( 'Banana Chips',    2,      2, '200 g pack',  90.00),  
( 'Bhujia Namkeen',  1,      2, '400 g pack', 110.00),  
( 'Paneer',          1,      3, '200 g block', 95.00),  
( 'Ghee',            1,      3, '1 L tin',    650.00),  
( 'Basmati Rice',    5,      4, '5 kg bag',   780.00),  
( 'Whole Wheat Atta', 1,      4, '10 kg bag',  460.00),  
( 'Garam Masala',    3,      5, '100 g box',  85.00),  
( 'Saffron',         3,      5, '1 g box',    399.00),  
( 'Kaju Katli',      4,      6, '500 g box',  540.00),  
( 'Gulab Jamun Mix', NULL,    6, '200 g pack',  75.00);
```

Step 5 - Customers table Our buyers across India, UK, Germany, UAE and USA. Some PostalCodes are NULL; Texas Tandoor and Kolkata Sweets never ordered.

```
CREATE TABLE Customers (  
    CustomerID      INT AUTO_INCREMENT PRIMARY KEY,  
    CustomerName    VARCHAR(100) NOT NULL,  
    ContactName     VARCHAR(100),  
    Address          VARCHAR(150),  
    City            VARCHAR(50),  
    PostalCode       VARCHAR(20),  
    Country          VARCHAR(50)  
);  
  
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country) VALUES  
( 'Sharma General Store', 'Rakesh Sharma', '12 MG Road',      'Ahmedabad', '380001', 'India'),  
( 'Patel Supermart',     'Nisha Patel', '45 CG Road',      'Ahmedabad', '380009', 'India'),  
( 'Mehta Kirana',        'Anil Mehta', '7 Station Road',  'Surat',     '395003', 'India'),  
( 'Fresh Basket',        'Priya Nair', '21 Brigade Road', 'Bengaluru', '560025', 'India'),  
( 'Desi Mart',           'Imran Khan', '88 Linking Road', 'Mumbai',    '400050', 'India'),  
( 'London Spice House',  'James Brown', '10 Brick Lane',   'London',    'E1 6QL',  'UK'),  
( 'Curry Corner',       'Sophie Clark', '5 Oxford Street', 'Manchester', 'M1 5AN',  'UK'),  
( 'Berlin Bazaar',       'Hans Mueller', 'Friedrichstr. 12', 'Berlin',    '10117',   'Germany'),  
( 'Munich Deli',        'Anna Schmidt', 'Marienplatz 4',   'Munich',    '80331',   'Germany'),  
( 'Dubai Gourmet',      'Omar Hassan', 'Sheikh Zayed Road', 'Dubai',     NULL,      'UAE'),  
( 'Sharjah Traders',    'Fatima Ali', 'Al Wahda Street', 'Sharjah',   NULL,      'UAE'),  
( 'New York Masala',     'David Lee', '120 Lexington Ave', 'New York',  '10016',   'USA'),  
( 'Texas Tandoor',      'Maria Garcia', '800 Main St',     'Houston',   '77002',   'USA'),  
( 'Kolkata Sweets',     'Sourav Das', '15 Park Street',  'Kolkata',   '700016',  'India');
```

Step 6 - Employees table Sales staff. ManagerID points back to the same table (used for SELF JOIN). Neha Joshi has not handled any order yet.

```
CREATE TABLE Employees (  
    EmployeeID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    BirthDate DATE,  
    Notes VARCHAR(255),  
    ManagerID INT NULL,  
    FOREIGN KEY (ManagerID) REFERENCES Employees(EmployeeID)  
);  
  
INSERT INTO Employees (FirstName, LastName, BirthDate, Notes, ManagerID) VALUES  
( 'Arjun', 'Mehta', '1980-05-14', 'Sales Head', NULL),  
( 'Kavya', 'Iyer', '1988-11-02', 'Senior sales, South region', 1),  
( 'Rohan', 'Desai', '1992-03-21', 'Sales executive, West region', 1),  
( 'Sara', 'Thomas', '1995-07-30', 'International accounts', 2),  
( 'Vikram', 'Singh', '1990-01-09', 'Sales executive, North region', 2),  
( 'Neha', 'Joshi', '1998-12-15', 'Trainee', 3);
```

Step 7 - Shippers table Courier companies that deliver orders.

```
CREATE TABLE Shippers (  
    ShipperID INT AUTO_INCREMENT PRIMARY KEY,  
    ShipperName VARCHAR(100) NOT NULL,  
    Phone VARCHAR(20)  
);  
  
INSERT INTO Shippers (ShipperName, Phone) VALUES  
( 'BlueDart Express', '1800-000-1001'),  
( 'Delhivery', '1800-000-1002'),  
( 'DHL Global', '1800-000-1003');
```

Step 8 - Orders table One row per order. Links a customer, the employee who handled it and the shipper. Order IDs start at 1001.

```
CREATE TABLE Orders (  
    OrderID INT AUTO_INCREMENT PRIMARY KEY,  
    CustomerID INT,  
    EmployeeID INT,  
    OrderDate DATE,  
    ShipperID INT,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID),  
    FOREIGN KEY (ShipperID) REFERENCES Shippers(ShipperID)  
) AUTO_INCREMENT = 1001;  
  
INSERT INTO Orders (CustomerID, EmployeeID, OrderDate, ShipperID) VALUES  
(1, 3, '2026-01-05', 2),  
(2, 3, '2026-01-12', 1),  
(6, 4, '2026-01-20', 3),  
(4, 2, '2026-02-03', 1),  
(5, 5, '2026-02-14', 2),  
(1, 3, '2026-02-25', 2),  
(8, 4, '2026-03-04', 3),  
(10, 4, '2026-03-15', 3),  
(3, 3, '2026-03-28', 1),  
(12, 4, '2026-04-06', 3),  
(2, 2, '2026-04-18', 1),  
(7, 4, '2026-05-02', 3),  
(9, 1, '2026-05-20', 3),  
(11, 5, '2026-06-01', 2),  
(5, 5, '2026-06-11', 2);
```

Step 9 - OrderDetails table The line items of each order: which product and how many. CHECK makes sure Quantity is always positive.

```
CREATE TABLE OrderDetails (  
    OrderDetailID INT AUTO_INCREMENT PRIMARY KEY,  
    OrderID INT,  
    ProductID INT,  
    Quantity INT CHECK (Quantity > 0),  
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),  
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
);  
  
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES  
(1001, 1, 10), (1001, 5, 20), (1001, 6, 15),  
(1002, 7, 5), (1002, 9, 8),  
(1003, 10, 50), (1003, 11, 10), (1003, 1, 25),  
(1004, 2, 12), (1004, 4, 30),  
(1005, 8, 6), (1005, 12, 4), (1005, 3, 24),  
(1006, 6, 20), (1006, 7, 3),  
(1007, 11, 5), (1007, 12, 10),  
(1008, 8, 40), (1008, 3, 60),  
(1009, 5, 15), (1009, 9, 10), (1009, 10, 20),  
(1010, 1, 30), (1010, 12, 15), (1010, 11, 8),  
(1011, 4, 25), (1011, 6, 10),  
(1012, 10, 35), (1012, 2, 20),  
(1013, 12, 6), (1013, 7, 4),  
(1014, 8, 25), (1014, 3, 48), (1014, 1, 12),  
(1015, 5, 40), (1015, 4, 40);
```

1.2 Look at every table

Run these after the setup script. Keep this chapter open as your reference while solving the problems - every answer in this book comes from these rows.

Problem 1.2.1 Show all categories.

SOLUTION (SQL)

```
SELECT * FROM Categories;
```

RESULT

CategoryID	CategoryName	Description
1	Beverages	Tea, coffee and juices
2	Snacks	Namkeen, chips and dry snacks
3	Dairy	Milk products
4	Grains	Rice, flour and cereals
5	Spices	Whole and ground spices
6	Sweets	Indian mithai and dessert mixes

6 rows in set

Problem 1.2.2 Show all suppliers.

SOLUTION (SQL)

SELECT * **FROM** Suppliers;

RESULT

SupplierID	SupplierName	ContactName	City	Country	Phone
1	Amul Traders	Mahesh Shah	Anand	India	+91-2692-000001
2	Himalaya Foods	Deepa Rawat	Dehradun	India	+91-135-0000002
3	Spice Route Ltd	Oliver Smith	London	UK	+44-20-00000003
4	Bavaria Imports	Klaus Weber	Munich	Germany	+49-89-00000004
5	Gulf Fresh LLC	Ahmed Saeed	Dubai	UAE	+971-4-00000005
6	Kerala Coconut Co	Thomas Mathew	Kochi	India	+91-484-0000006

6 rows in set

Problem 1.2.3 Show all products.

SOLUTION (SQL)

SELECT * **FROM** Products;

RESULT

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Masala Chai	2	1	500 g pack	250.00
2	Filter Coffee	2	1	250 g pack	320.00
3	Mango Juice	5	1	1 L bottle	120.00
4	Banana Chips	2	2	200 g pack	90.00
5	Bhujia Namkeen	1	2	400 g pack	110.00
6	Paneer	1	3	200 g block	95.00
7	Ghee	1	3	1 L tin	650.00
8	Basmati Rice	5	4	5 kg bag	780.00
9	Whole Wheat Atta	1	4	10 kg bag	460.00
10	Garam Masala	3	5	100 g box	85.00
11	Saffron	3	5	1 g box	399.00
12	Kaju Katli	4	6	500 g box	540.00
13	Gulab Jamun Mix	NULL	6	200 g pack	75.00

13 rows in set

Problem 1.2.4 Show all customers.

SOLUTION (SQL)

SELECT * **FROM** Customers;

RESULT

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Sharma General Store	Rakesh Sharma	12 MG Road	Ahmedabad	380001	India
2	Patel Supermart	Nisha Patel	45 CG Road	Ahmedabad	380009	India
3	Mehta Kirana	Anil Mehta	7 Station Road	Surat	395003	India
4	Fresh Basket	Priya Nair	21 Brigade Road	Bengaluru	560025	India
5	Desi Mart	Imran Khan	88 Linking Road	Mumbai	400050	India
6	London Spice House	James Brown	10 Brick Lane	London	E1 6QL	UK
7	Curry Corner	Sophie Clark	5 Oxford Street	Manchester	M1 5AN	UK
8	Berlin Bazaar	Hans Mueller	Friedrichstr. 12	Berlin	10117	Germany
9	Munich Deli	Anna Schmidt	Marienplatz 4	Munich	80331	Germany
10	Dubai Gourmet	Omar Hassan	Sheikh Zayed Road	Dubai	NULL	UAE
11	Sharjah Traders	Fatima Ali	Al Wahda Street	Sharjah	NULL	UAE
12	New York Masala	David Lee	120 Lexington Ave	New York	10016	USA
13	Texas Tandoor	Maria Garcia	800 Main St	Houston	77002	USA
14	Kolkata Sweets	Sourav Das	15 Park Street	Kolkata	700016	India

14 rows in set

Problem 1.2.5 Show all employees.

SOLUTION (SQL)

```
SELECT * FROM Employees;
```

RESULT

EmployeeID	FirstName	LastName	BirthDate	Notes	ManagerID
1	Arjun	Mehta	1980-05-14	Sales Head	NULL
2	Kavya	Iyer	1988-11-02	Senior sales, South region	1
3	Rohan	Desai	1992-03-21	Sales executive, West region	1
4	Sara	Thomas	1995-07-30	International accounts	2
5	Vikram	Singh	1990-01-09	Sales executive, North region	2
6	Neha	Joshi	1998-12-15	Trainee	3

6 rows in set

Problem 1.2.6 Show all shippers.

SOLUTION (SQL)

```
SELECT * FROM Shippers;
```

RESULT

ShipperID	ShipperName	Phone
1	BlueDart Express	1800-000-1001
2	Delhivery	1800-000-1002
3	DHL Global	1800-000-1003

3 rows in set

Problem 1.2.7 Show all orders.

SOLUTION (SQL)

```
SELECT * FROM Orders;
```

RESULT

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
1001	1	3	2026-01-05	2
1002	2	3	2026-01-12	1
1003	6	4	2026-01-20	3
1004	4	2	2026-02-03	1
1005	5	5	2026-02-14	2
1006	1	3	2026-02-25	2
1007	8	4	2026-03-04	3
1008	10	4	2026-03-15	3
1009	3	3	2026-03-28	1
1010	12	4	2026-04-06	3
1011	2	2	2026-04-18	1
1012	7	4	2026-05-02	3
1013	9	1	2026-05-20	3
1014	11	5	2026-06-01	2
1015	5	5	2026-06-11	2

15 rows in set

Problem 1.2.8 Show all order line items.

SOLUTION (SQL)

```
SELECT * FROM OrderDetails;
```

RESULT

OrderDetailID	OrderID	ProductID	Quantity
1	1001	1	10
2	1001	5	20
3	1001	6	15
4	1002	7	5
5	1002	9	8
6	1003	10	50
7	1003	11	10
8	1003	1	25
9	1004	2	12
10	1004	4	30
11	1005	8	6
12	1005	12	4
13	1005	3	24
14	1006	6	20
15	1006	7	3
16	1007	11	5
17	1007	12	10
18	1008	8	40
19	1008	3	60
20	1009	5	15
21	1009	9	10
22	1009	10	20
23	1010	1	30
24	1010	12	15
25	1010	11	8
26	1011	4	25
27	1011	6	10
28	1012	10	35
29	1012	2	20
30	1013	12	6
31	1013	7	4
32	1014	8	25
33	1014	3	48
34	1014	1	12
35	1015	5	40
36	1015	4	40

36 rows in set

Problem 1.2.9 List every table in the database.

SOLUTION (SQL)

```
SHOW TABLES;
```

RESULT

Tables_in_ShopDB
Categories
Customers
Employees
OrderDetails
Orders
Products
Shippers
Suppliers

8 rows in set

Problem 1.2.10 See the structure (columns, types, keys) of the Products table.

SOLUTION (SQL)

```
DESCRIBE Products;
```

RESULT

Field	Type	Null	Key	Default	Extra
ProductID	int(11)	NO	PRI	NULL	auto_increment
ProductName	varchar(100)	NO		NULL	
SupplierID	int(11)	YES	MUL	NULL	
CategoryID	int(11)	YES	MUL	NULL	
Unit	varchar(50)	YES		NULL	
Price	decimal(10,2)	YES		0.00	

6 rows in set

SQL Tutorial - Querying & Changing Data

SELECT to Operators: every topic in the W3Schools 'SQL Tutorial' section

2.1 SQL Syntax & SELECT

SELECT retrieves data from one or more tables. SQL keywords are not case-sensitive (select = SELECT), and each statement ends with a semicolon.

SYNTAX

```
SELECT column1, column2, ... FROM table_name;
SELECT * FROM table_name;
```

Problem 2.1.1 The marketing team needs a list of customer names and their cities.

SOLUTION (SQL)

```
SELECT CustomerName, City
FROM Customers;
```

RESULT

CustomerName	City
Sharma General Store	Ahmedabad
Patel Supermart	Ahmedabad
Mehta Kirana	Surat
Fresh Basket	Bengaluru
Desi Mart	Mumbai
London Spice House	London
Curry Corner	Manchester
Berlin Bazaar	Berlin
Munich Deli	Munich
Dubai Gourmet	Dubai
Sharjah Traders	Sharjah
New York Masala	New York
Texas Tandoor	Houston
Kolkata Sweets	Kolkata

14 rows in set

Problem 2.1.2 Show product name, unit and price for every product.

SOLUTION (SQL)

```
SELECT ProductName, Unit, Price
FROM Products;
```

RESULT

ProductName	Unit	Price
Masala Chai	500 g pack	250.00
Filter Coffee	250 g pack	320.00
Mango Juice	1 L bottle	120.00
Banana Chips	200 g pack	90.00
Bhujia Namkeen	400 g pack	110.00
Paneer	200 g block	95.00
Ghee	1 L tin	650.00
Basmati Rice	5 kg bag	780.00
Whole Wheat Atta	10 kg bag	460.00
Garam Masala	100 g box	85.00
Saffron	1 g box	399.00
Kaju Katli	500 g box	540.00
Gulab Jamun Mix	200 g pack	75.00

13 rows in set

2.2 SELECT DISTINCT

DISTINCT removes duplicate values and returns only unique ones.

SYNTAX

```
SELECT DISTINCT column1, column2 FROM table_name;
```

Problem 2.2.1 Which countries do our customers come from? Show each country only once.

SOLUTION (SQL)

```
SELECT DISTINCT Country
FROM Customers;
```

RESULT

```
+-----+
| Country |
+-----+
| India   |
| UK      |
| Germany |
| UAE     |
| USA     |
+-----+
5 rows in set
```

Problem 2.2.2 How many different countries do we sell to?

SOLUTION (SQL)

```
SELECT COUNT(DISTINCT Country) AS NumberOfCountries
FROM Customers;
```

RESULT

```
+-----+
| NumberOfCountries |
+-----+
| 5 |
+-----+
1 row in set
```

Problem 2.2.3 Which unique city + country combinations do suppliers come from?

SOLUTION (SQL)

```
SELECT DISTINCT City, Country
FROM Suppliers;
```

RESULT

```
+-----+-----+
| City   | Country |
+-----+-----+
| Anand  | India   |
| Dehradun | India   |
| London | UK      |
| Munich | Germany |
| Dubai  | UAE     |
| Kochi  | India   |
+-----+-----+
6 rows in set
```

2.3 WHERE

WHERE filters rows so only records that satisfy a condition are returned. Text values go in single quotes; numbers do not.

SYNTAX

```
SELECT column1, column2 FROM table_name
WHERE condition;
```

Problem 2.3.1 Find all customers located in India.

SOLUTION (SQL)

```
SELECT * FROM Customers
WHERE Country = 'India';
```

RESULT

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Sharma General Store	Rakesh Sharma	12 MG Road	Ahmedabad	380001	India
2	Patel Supermart	Nisha Patel	45 CG Road	Ahmedabad	380009	India
3	Mehta Kirana	Anil Mehta	7 Station Road	Surat	395003	India
4	Fresh Basket	Priya Nair	21 Brigade Road	Bengaluru	560025	India
5	Desi Mart	Imran Khan	88 Linking Road	Mumbai	400050	India
14	Kolkata Sweets	Sourav Das	15 Park Street	Kolkata	700016	India

6 rows in set

Problem 2.3.2 Get the product with ProductID 8.

SOLUTION (SQL)

```
SELECT * FROM Products
WHERE ProductID = 8;
```

RESULT

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
8	Basmati Rice	5	4	5 kg bag	780.00

1 row in set

Problem 2.3.3 Which products cost more than 400?

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE Price > 400;
```

RESULT

ProductName	Price
Ghee	650.00
Basmati Rice	780.00
Whole Wheat Atta	460.00
Kaju Katli	540.00

4 rows in set

Problem 2.3.4 Show orders placed on or after 1 April 2026.

SOLUTION (SQL)

```
SELECT * FROM Orders
WHERE OrderDate >= '2026-04-01';
```

RESULT

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
1010	12	4	2026-04-06	3
1011	2	2	2026-04-18	1
1012	7	4	2026-05-02	3
1013	9	1	2026-05-20	3
1014	11	5	2026-06-01	2
1015	5	5	2026-06-11	2

6 rows in set

Problem 2.3.5 Show all customers that are NOT in India using the <> operator.

SOLUTION (SQL)

```
SELECT CustomerName, Country
FROM Customers
WHERE Country <> 'India';
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| London Spice House | UK      |
| Curry Corner      | UK      |
| Berlin Bazaar      | Germany |
| Munich Deli        | Germany |
| Dubai Gourmet      | UAE     |
| Sharjah Traders    | UAE     |
| New York Masala     | USA     |
| Texas Tandoor      | USA     |
+-----+-----+
8 rows in set
```

Tip: Operators usable in WHERE: =, >, <, >=, <=, <> (or !=), BETWEEN, LIKE, IN.

2.4 ORDER BY

ORDER BY sorts the result. ASC (ascending) is the default; DESC sorts in reverse.

SYNTAX

```
SELECT column1, column2 FROM table_name
ORDER BY column1 ASC|DESC, column2 ASC|DESC;
```

Problem 2.4.1 List products from cheapest to most expensive.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
ORDER BY Price;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Gulab Jamun Mix | 75.00 |
| Garam Masala   | 85.00 |
| Banana Chips   | 90.00 |
| Paneer         | 95.00 |
| Bhujia Namkeen | 110.00 |
| Mango Juice    | 120.00 |
| Masala Chai    | 250.00 |
| Filter Coffee  | 320.00 |
| Saffron        | 399.00 |
| Whole Wheat Atta | 460.00 |
| Kaju Katli     | 540.00 |
| Ghee           | 650.00 |
| Basmati Rice   | 780.00 |
+-----+-----+
13 rows in set
```

Problem 2.4.2 List products from most expensive to cheapest.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
ORDER BY Price DESC;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Basmati Rice | 780.00 |
| Ghee         | 650.00 |
| Kaju Katli   | 540.00 |
| Whole Wheat Atta | 460.00 |
| Saffron      | 399.00 |
| Filter Coffee | 320.00 |
| Masala Chai  | 250.00 |
| Mango Juice  | 120.00 |
| Bhujia Namkeen | 110.00 |
| Paneer       | 95.00  |
| Banana Chips | 90.00  |
| Garam Masala | 85.00  |
| Gulab Jamun Mix | 75.00  |
+-----+-----+
13 rows in set
```

Problem 2.4.3 Sort customers alphabetically by name (text sorts A-Z).

SOLUTION (SQL)

```
SELECT CustomerName, City
FROM Customers
ORDER BY CustomerName;
```

RESULT

```
+-----+-----+
| CustomerName | City |
+-----+-----+
| Berlin Bazaar | Berlin |
| Curry Corner  | Manchester |
| Desi Mart     | Mumbai |
| Dubai Gourmet | Dubai |
| Fresh Basket  | Bengaluru |
| Kolkata Sweets | Kolkata |
| London Spice House | London |
| Mehta Kirana  | Surat |
| Munich Deli   | Munich |
| New York Masala | New York |
| Patel Supermart | Ahmedabad |
| Sharjah Traders | Sharjah |
| Sharma General Store | Ahmedabad |
| Texas Tandoor | Houston |
+-----+-----+
14 rows in set
```

Problem 2.4.4 Sort customers by Country, and inside each country by CustomerName in reverse order.

SOLUTION (SQL)

```
SELECT Country, CustomerName
FROM Customers
ORDER BY Country ASC, CustomerName DESC;
```

RESULT

```
+-----+-----+
| Country | CustomerName |
+-----+-----+
| Germany | Munich Deli  |
| Germany | Berlin Bazaar|
| India   | Sharma General Store |
| India   | Patel Supermart |
| India   | Mehta Kirana  |
| India   | Kolkata Sweets|
| India   | Fresh Basket  |
| India   | Desi Mart     |
| UAE     | Sharjah Traders|
| UAE     | Dubai Gourmet |
| UK       | London Spice House|
| UK       | Curry Corner  |
| USA     | Texas Tandoor |
| USA     | New York Masala|
+-----+-----+
14 rows in set
```

2.5 AND

AND returns a row only when ALL conditions are true.

SYNTAX

```
SELECT * FROM table_name
WHERE condition1 AND condition2 AND condition3;
```

Problem 2.5.1 Find customers in India whose city is Ahmedabad.

SOLUTION (SQL)

```
SELECT * FROM Customers
WHERE Country = 'India' AND City = 'Ahmedabad';
```

RESULT

```
+-----+-----+-----+-----+-----+-----+-----+
| CustomerID | CustomerName | ContactName | Address | City | PostalCode | Country |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Sharma General Store | Rakesh Sharma | 12 MG Road | Ahmedabad | 380001 | India |
| 2 | Patel Supermart | Nisha Patel | 45 CG Road | Ahmedabad | 380009 | India |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set
```

Problem 2.5.2 Find dairy products (CategoryID 3) that cost less than 100.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE CategoryID = 3 AND Price < 100;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Paneer      | 95.00 |
+-----+-----+
1 row in set
```

Problem 2.5.3 Find Indian customers who are in Surat OR Mumbai (combine AND with OR using brackets).

SOLUTION (SQL)

```
SELECT CustomerName, City
FROM Customers
WHERE Country = 'India' AND (City = 'Surat' OR City = 'Mumbai');
```

RESULT

```
+-----+-----+
| CustomerName | City |
+-----+-----+
| Mehta Kirana | Surat |
| Desi Mart    | Mumbai |
+-----+-----+
2 rows in set
```

Tip: Without the brackets, AND is evaluated before OR and you would get a different (wrong) result.

2.6 OR

OR returns a row when ANY of the conditions is true.

SYNTAX

```
SELECT * FROM table_name
WHERE condition1 OR condition2;
```

Problem 2.6.1 Show customers from the UK or the UAE.

SOLUTION (SQL)

```
SELECT CustomerName, Country
FROM Customers
WHERE Country = 'UK' OR Country = 'UAE';
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| London Spice House | UK |
| Curry Corner | UK |
| Dubai Gourmet | UAE |
| Sharjah Traders | UAE |
+-----+-----+
4 rows in set
```

Problem 2.6.2 Show customers who are in Ahmedabad OR whose name starts with 'D'.

SOLUTION (SQL)

```
SELECT CustomerName, City
FROM Customers
WHERE City = 'Ahmedabad' OR CustomerName LIKE 'D%';
```

RESULT

```
+-----+-----+
| CustomerName | City |
+-----+-----+
| Sharma General Store | Ahmedabad |
| Patel Supermart | Ahmedabad |
| Desi Mart | Mumbai |
| Dubai Gourmet | Dubai |
+-----+-----+
4 rows in set
```

2.7 NOT

NOT reverses a condition. It can be combined with =, LIKE, BETWEEN, IN, >, < etc.

SYNTAX

```
SELECT * FROM table_name
WHERE NOT condition;
```

Problem 2.7.1 List customers that are not from India.

SOLUTION (SQL)

```
SELECT CustomerName, Country
FROM Customers
WHERE NOT Country = 'India';
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| London Spice House | UK      |
| Curry Corner      | UK      |
| Berlin Bazaar     | Germany |
| Munich Deli       | Germany |
| Dubai Gourmet     | UAE     |
| Sharjah Traders   | UAE     |
| New York Masala    | USA     |
| Texas Tandoor     | USA     |
+-----+-----+
8 rows in set
```

Problem 2.7.2 Customers whose name does NOT start with 'S'.

SOLUTION (SQL)

```
SELECT CustomerName
FROM Customers
WHERE CustomerName NOT LIKE 'S%';
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Patel Supermart |
| Mehta Kirana   |
| Fresh Basket   |
| Desi Mart      |
| London Spice House |
| Curry Corner   |
| Berlin Bazaar  |
| Munich Deli    |
| Dubai Gourmet  |
| New York Masala |
| Texas Tandoor  |
| Kolkata Sweets |
+-----+
12 rows in set
```

Problem 2.7.3 Products whose price is NOT between 100 and 500.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE Price NOT BETWEEN 100 AND 500;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Banana Chips | 90.00 |
| Paneer       | 95.00 |
| Ghee         | 650.00 |
| Basmati Rice | 780.00 |
| Garam Masala | 85.00 |
| Kaju Katli   | 540.00 |
| Gulab Jamun Mix | 75.00 |
+-----+-----+
7 rows in set
```

Problem 2.7.4 Customers not located in India, UK or USA.

SOLUTION (SQL)

```
SELECT CustomerName, Country
FROM Customers
WHERE Country NOT IN ('India', 'UK', 'USA');
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| Berlin Bazaar | Germany |
| Munich Deli   | Germany |
| Dubai Gourmet | UAE     |
| Sharjah Traders | UAE     |
+-----+-----+
4 rows in set
```

Problem 2.7.5 Products whose price is NOT greater than 100.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE NOT Price > 100;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Banana Chips | 90.00 |
| Paneer       | 95.00 |
| Garam Masala | 85.00 |
| Gulab Jamun Mix | 75.00 |
+-----+-----+
4 rows in set
```

2.8 INSERT INTO

INSERT INTO adds new rows. You can supply all columns or only some; columns you skip get NULL (or their DEFAULT). AUTO_INCREMENT columns such as CustomerID are filled automatically.

SYNTAX

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Problem 2.8.1 A new customer 'Jaipur Rasoi' has signed up. Add them with full details and verify.

SOLUTION (SQL)

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
VALUES ('Jaipur Rasoi', 'Kiran Rathore', '3 MI Road', 'Jaipur', '302001', 'India');

SELECT * FROM Customers WHERE CustomerName = 'Jaipur Rasoi';
```

RESULT

Query OK, 1 row affected

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
15	Jaipur Rasoi	Kiran Rathore	3 MI Road	Jaipur	302001	India

1 row in set

Problem 2.8.2 Another customer 'Pune Pantry' gave only name, city and country. Insert only those columns.

SOLUTION (SQL)

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Pune Pantry', 'Pune', 'India');

SELECT * FROM Customers WHERE CustomerName = 'Pune Pantry';
```

RESULT

Query OK, 1 row affected

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
16	Pune Pantry	NULL	NULL	Pune	NULL	India

1 row in set

Tip: ContactName, Address and PostalCode are NULL because we did not provide them.

Problem 2.8.3 Add two new shippers in a single statement.

SOLUTION (SQL)

```
INSERT INTO Shippers (ShipperName, Phone)
VALUES ('India Post', '1800-000-1004'),
       ('FedEx India', '1800-000-1005');

SELECT * FROM Shippers;
```

RESULT

Query OK, 2 rows affected
Records: 2 Duplicates: 0 Warnings: 0

ShipperID	ShipperName	Phone
1	BlueDart Express	1800-000-1001
2	Delhivery	1800-000-1002
3	DHL Global	1800-000-1003
4	India Post	1800-000-1004
5	FedEx India	1800-000-1005

5 rows in set

2.9 NULL Values

NULL means 'no value'. You cannot test it with = or <>; always use IS NULL or IS NOT NULL.

SYNTAX

```
SELECT column_names FROM table_name WHERE column_name IS NULL;
SELECT column_names FROM table_name WHERE column_name IS NOT NULL;
```

Problem 2.9.1 Which customers have no postal code recorded?

SOLUTION (SQL)

```
SELECT CustomerName, City, PostalCode
FROM Customers
WHERE PostalCode IS NULL;
```

RESULT

```
+-----+-----+-----+
| CustomerName | City   | PostalCode |
+-----+-----+-----+
| Dubai Gourmet | Dubai  | NULL       |
| Sharjah Traders | Sharjah | NULL       |
| Pune Pantry    | Pune   | NULL       |
+-----+-----+-----+
3 rows in set
```

Problem 2.9.2 Show customers that DO have a contact name.

SOLUTION (SQL)

```
SELECT CustomerName, ContactName
FROM Customers
WHERE ContactName IS NOT NULL;
```

RESULT

```
+-----+-----+
| CustomerName | ContactName |
+-----+-----+
| Sharma General Store | Rakesh Sharma |
| Patel Supermart    | Nisha Patel  |
| Mehta Kirana       | Anil Mehta   |
| Fresh Basket       | Priya Nair   |
| Desi Mart          | Imran Khan   |
| London Spice House | James Brown   |
| Curry Corner       | Sophie Clark  |
| Berlin Bazaar      | Hans Mueller  |
| Munich Deli        | Anna Schmidt  |
| Dubai Gourmet      | Omar Hassan   |
| Sharjah Traders    | Fatima Ali    |
| New York Masala    | David Lee     |
| Texas Tandoor      | Maria Garcia  |
| Kolkata Sweets     | Sourav Das    |
| Jaipur Rasoi       | Kiran Rathore |
+-----+-----+
15 rows in set
```

Problem 2.9.3 Find products that have no supplier assigned.

SOLUTION (SQL)

```
SELECT ProductName, SupplierID
FROM Products
WHERE SupplierID IS NULL;
```

RESULT

```
+-----+-----+
| ProductName | SupplierID |
+-----+-----+
| Gulab Jamun Mix | NULL      |
+-----+-----+
1 row in set
```

Problem 2.9.4 Common mistake: comparing with = NULL returns nothing, even though NULL rows exist.

SOLUTION (SQL)

```
SELECT CustomerName FROM Customers
WHERE PostalCode = NULL;
```

RESULT

Empty set

2.10 UPDATE

UPDATE changes existing rows. The WHERE clause decides WHICH rows change - forget it and every row in the table is updated!

SYNTAX

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Problem 2.10.1 Pune Pantry sent their contact person, address and PIN code. Update the record.

SOLUTION (SQL)

```
UPDATE Customers
SET ContactName = 'Rahul Joshi', Address = '9 FC Road', PostalCode = '411004'
WHERE CustomerName = 'Pune Pantry';

SELECT * FROM Customers WHERE CustomerName = 'Pune Pantry';
```

RESULT

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1  Warnings: 0

+-----+-----+-----+-----+-----+-----+-----+
| CustomerID | CustomerName | ContactName | Address | City | PostalCode | Country |
+-----+-----+-----+-----+-----+-----+-----+
|          16 | Pune Pantry | Rahul Joshi | 9 FC Road | Pune | 411004 | India |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set
```

Problem 2.10.2 Spice prices (CategoryID 5) increase by 10%. Update all of them at once.

SOLUTION (SQL)

```
UPDATE Products
SET Price = Price * 1.10
WHERE CategoryID = 5;

SELECT ProductName, Price FROM Products WHERE CategoryID = 5;
```

RESULT

```
Query OK, 2 rows affected
Rows matched: 2  Changed: 2  Warnings: 0

+-----+-----+
| ProductName | Price |
+-----+-----+
| Garam Masala | 93.50 |
| Saffron      | 438.90 |
+-----+-----+
2 rows in set
```

Problem 2.10.3 DANGER - do NOT run: an UPDATE without WHERE changes every row.

SOLUTION (SQL)

```
UPDATE Customers
SET Country = 'India'; -- every customer would become Indian!
```

NOTE

(Not executed. Always write the WHERE clause first, or test it with a SELECT.)

2.11 DELETE

DELETE removes rows. As with UPDATE, the WHERE clause decides which rows are removed.

SYNTAX

```
DELETE FROM table_name WHERE condition;
```

Problem 2.11.1 Jaipur Rasoi cancelled their account. Delete the customer.

SOLUTION (SQL)

```
DELETE FROM Customers
WHERE CustomerName = 'Jaipur Rasoi';

SELECT COUNT(*) AS RemainingCustomers FROM Customers;
```

RESULT

Query OK, 1 row affected

```
+-----+
| RemainingCustomers |
+-----+
|                15 |
+-----+
1 row in set
```

Problem 2.11.2 Remove the two shippers we added earlier (ShipperID above 3).

SOLUTION (SQL)

```
DELETE FROM Shippers
WHERE ShipperID > 3;

SELECT * FROM Shippers;
```

RESULT

Query OK, 2 rows affected

```
+-----+-----+-----+
| ShipperID | ShipperName | Phone |
+-----+-----+-----+
| 1 | BlueDart Express | 1800-000-1001 |
| 2 | Delhivery | 1800-000-1002 |
| 3 | DHL Global | 1800-000-1003 |
+-----+-----+-----+
3 rows in set
```

Problem 2.11.3 DANGER - do NOT run: delete all rows but keep the table structure.

SOLUTION (SQL)

```
DELETE FROM Customers;
```

NOTE

(Not executed. This would delete every customer. TRUNCATE TABLE does the same faster - see Chapter 3.)

2.12 SELECT TOP / LIMIT / FETCH FIRST

Returns only the first N rows. Syntax depends on the database: MySQL/PostgreSQL use LIMIT, SQL Server/MS Access use TOP, Oracle uses FETCH FIRST (or ROWNUM in old versions).

SYNTAX

```
-- MySQL
SELECT columns FROM table_name WHERE condition LIMIT number;
-- SQL Server / MS Access
SELECT TOP number columns FROM table_name;
-- Oracle 12c+
SELECT columns FROM table_name FETCH FIRST number ROWS ONLY;
-- Oracle (old)
SELECT columns FROM table_name WHERE ROWNUM <= number;
```

Problem 2.12.1 Show the 3 most expensive products.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
ORDER BY Price DESC
LIMIT 3;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Basmati Rice | 780.00 |
| Ghee         | 650.00 |
| Kaju Katli   | 540.00 |
+-----+-----+
3 rows in set
```

Tip: SQL Server: SELECT TOP 3 ProductName, Price FROM Products ORDER BY Price DESC;

Problem 2.12.2 Pagination: show page 2 of the customer list when each page holds 5 customers.

SOLUTION (SQL)

```
SELECT CustomerID, CustomerName
FROM Customers
ORDER BY CustomerID
LIMIT 5 OFFSET 5;
```

RESULT

```
+-----+-----+
| CustomerID | CustomerName |
+-----+-----+
| 6          | London Spice House |
| 7          | Curry Corner      |
| 8          | Berlin Bazaar     |
| 9          | Munich Deli       |
| 10         | Dubai Gourmet     |
+-----+-----+
5 rows in set
```

Problem 2.12.3 Show only the first 2 customers from the UK.

SOLUTION (SQL)

```
SELECT CustomerName, City
FROM Customers
WHERE Country = 'UK'
LIMIT 2;
```

RESULT

```
+-----+-----+
| CustomerName | City      |
+-----+-----+
| London Spice House | London  |
| Curry Corner      | Manchester |
+-----+-----+
2 rows in set
```

Problem 2.12.4 SQL Server version of 'top 50 percent' (reference only).

SOLUTION (SQL)

```
SELECT TOP 50 PERCENT * FROM Customers;
```

NOTE

(SQL Server / MS Access only - MySQL has no PERCENT option.)

2.13 Aggregate Functions

Aggregate functions calculate one value from many rows: MIN(), MAX(), COUNT(), SUM(), AVG(). They ignore NULL values (except COUNT(*)). They are often used with GROUP BY.

Problem 2.13.1 Give a one-line price summary of the whole product catalogue.

SOLUTION (SQL)

```
SELECT MIN(Price) AS Cheapest,  
       MAX(Price) AS Costliest,  
       COUNT(*)   AS TotalProducts,  
       SUM(Price) AS SumOfPrices,  
       AVG(Price) AS AveragePrice  
FROM Products;
```

RESULT

```
+-----+-----+-----+-----+-----+  
| Cheapest | Costliest | TotalProducts | SumOfPrices | AveragePrice |  
+-----+-----+-----+-----+-----+  
| 75.00 | 780.00 | 13 | 4022.40 | 309.415385 |  
+-----+-----+-----+-----+-----+  
1 row in set
```

2.14 MIN() and MAX()

MIN() returns the smallest value, MAX() the largest. They also work on text (alphabetical) and dates.

SYNTAX

```
SELECT MIN(column_name) FROM table_name WHERE condition;  
SELECT MAX(column_name) FROM table_name WHERE condition;
```

Problem 2.14.1 What is the lowest product price?

SOLUTION (SQL)

```
SELECT MIN(Price) AS SmallestPrice  
FROM Products;
```

RESULT

```
+-----+  
| SmallestPrice |  
+-----+  
| 75.00 |  
+-----+  
1 row in set
```

Problem 2.14.2 What is the highest product price?

SOLUTION (SQL)

```
SELECT MAX(Price) AS LargestPrice  
FROM Products;
```

RESULT

```
+-----+  
| LargestPrice |  
+-----+  
| 780.00 |  
+-----+  
1 row in set
```

Problem 2.14.3 When was our first order and our latest order?

SOLUTION (SQL)

```
SELECT MIN(OrderDate) AS FirstOrder,  
       MAX(OrderDate) AS LatestOrder  
FROM Orders;
```

RESULT

```
+-----+-----+  
| FirstOrder | LatestOrder |  
+-----+-----+  
| 2026-01-05 | 2026-06-11 |  
+-----+-----+  
1 row in set
```

Problem 2.14.4 Find the cheapest price in each category.

SOLUTION (SQL)

```
SELECT CategoryID, MIN(Price) AS CheapestInCategory  
FROM Products  
GROUP BY CategoryID;
```

RESULT

```
+-----+-----+  
| CategoryID | CheapestInCategory |  
+-----+-----+  
|          1 |          120.00 |  
|          2 |           90.00 |  
|          3 |           95.00 |  
|          4 |          460.00 |  
|          5 |           93.50 |  
|          6 |           75.00 |  
+-----+-----+  
6 rows in set
```

2.15 COUNT()

COUNT(*) counts rows. COUNT(column) counts only non-NULL values in that column.

SYNTAX

```
SELECT COUNT(column_name) FROM table_name WHERE condition;
```

Problem 2.15.1 How many products do we sell?

SOLUTION (SQL)

```
SELECT COUNT(*) AS TotalProducts  
FROM Products;
```

RESULT

```
+-----+  
| TotalProducts |  
+-----+  
|          13 |  
+-----+  
1 row in set
```

Problem 2.15.2 How many products cost more than 200?

SOLUTION (SQL)

```
SELECT COUNT(ProductID) AS PremiumProducts  
FROM Products  
WHERE Price > 200;
```

RESULT

```
+-----+  
| PremiumProducts |  
+-----+  
|          7 |  
+-----+  
1 row in set
```

Problem 2.15.3 Compare COUNT(*) and COUNT(PostalCode) to see how NULLs are ignored.

SOLUTION (SQL)

```
SELECT COUNT(*)           AS AllCustomers,
       COUNT(PostalCode) AS WithPostalCode
FROM Customers;
```

RESULT

```
+-----+-----+
| AllCustomers | WithPostalCode |
+-----+-----+
|          15 |          13 |
+-----+-----+
1 row in set
```

Problem 2.15.4 How many orders did each employee handle?

SOLUTION (SQL)

```
SELECT EmployeeID, COUNT(*) AS OrdersHandled
FROM Orders
GROUP BY EmployeeID;
```

RESULT

```
+-----+-----+
| EmployeeID | OrdersHandled |
+-----+-----+
|          1 |          1 |
|          2 |          2 |
|          3 |          4 |
|          4 |          5 |
|          5 |          3 |
+-----+-----+
5 rows in set
```

2.16 SUM()

SUM() adds up a numeric column or expression.

SYNTAX

```
SELECT SUM(column_name) FROM table_name WHERE condition;
```

Problem 2.16.1 What is the total quantity of all items ever ordered?

SOLUTION (SQL)

```
SELECT SUM(Quantity) AS TotalItemsOrdered
FROM OrderDetails;
```

RESULT

```
+-----+
| TotalItemsOrdered |
+-----+
|          720 |
+-----+
1 row in set
```

Problem 2.16.2 How many units of Basmati Rice (ProductID 8) have been sold?

SOLUTION (SQL)

```
SELECT SUM(Quantity) AS BasmatiUnitsSold
FROM OrderDetails
WHERE ProductID = 8;
```

RESULT

```
+-----+
| BasmatiUnitsSold |
+-----+
|          71 |
+-----+
1 row in set
```

Problem 2.16.3 What is the total revenue of the company? (quantity x price of every line item)

SOLUTION (SQL)

```
SELECT SUM(od.Quantity * p.Price) AS TotalRevenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID;
```

RESULT

```
+-----+
| TotalRevenue |
+-----+
|    176677.20 |
+-----+
1 row in set
```

2.17 AVG()

AVG() returns the average value of a numeric column. NULLs are ignored.

SYNTAX

```
SELECT AVG(column_name) FROM table_name WHERE condition;
```

Problem 2.17.1 What is the average product price?

SOLUTION (SQL)

```
SELECT AVG(Price) AS AveragePrice
FROM Products;
```

RESULT

```
+-----+
| AveragePrice |
+-----+
|    309.415385 |
+-----+
1 row in set
```

Problem 2.17.2 List products priced above the average price.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products)
ORDER BY Price DESC;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Basmati Rice | 780.00 |
| Ghee         | 650.00 |
| Kaju Katli   | 540.00 |
| Whole Wheat Atta | 460.00 |
| Saffron      | 438.90 |
| Filter Coffee | 320.00 |
+-----+-----+
6 rows in set
```

Problem 2.17.3 What is the average quantity per order line, rounded to 2 decimals?

SOLUTION (SQL)

```
SELECT ROUND(AVG(Quantity), 2) AS AvgQtyPerLine
FROM OrderDetails;
```

RESULT

```
+-----+
| AvgQtyPerLine |
+-----+
|          20.00 |
+-----+
1 row in set
```

2.18 LIKE

LIKE searches for a pattern in text. % = any number of characters (including zero), _ = exactly one character.

SYNTAX

```
SELECT columns FROM table_name WHERE column LIKE pattern;
```

Problem 2.18.1 Customers whose name starts with 'S'.

SOLUTION (SQL)

```
SELECT CustomerName FROM Customers
WHERE CustomerName LIKE 'S%';
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Sharma General Store |
| Sharjah Traders |
+-----+
2 rows in set
```

Problem 2.18.2 Customers whose name ends with 'Mart'.

SOLUTION (SQL)

```
SELECT CustomerName FROM Customers
WHERE CustomerName LIKE '%Mart';
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Patel Supermart |
| Desi Mart |
+-----+
2 rows in set
```

Problem 2.18.3 Customers whose name contains the word 'Spice'.

SOLUTION (SQL)

```
SELECT CustomerName FROM Customers
WHERE CustomerName LIKE '%Spice%';
```

RESULT

```
+-----+
| CustomerName |
+-----+
| London Spice House |
+-----+
1 row in set
```

Problem 2.18.4 Cities whose second letter is 'u'.

SOLUTION (SQL)

```
SELECT DISTINCT City FROM Customers
WHERE City LIKE '_u%';
```

RESULT

```
+-----+
| City |
+-----+
| Surat |
| Mumbai |
| Munich |
| Dubai |
| Pune |
+-----+
5 rows in set
```

Problem 2.18.5 Products that start with 'M' and end with 'i'.

SOLUTION (SQL)

```
SELECT ProductName FROM Products
WHERE ProductName LIKE 'M%i';
```

RESULT

```
+-----+
| ProductName |
+-----+
| Masala Chai |
+-----+
1 row in set
```

2.19 Wildcards

Wildcards are the special characters used with LIKE. MySQL supports % and _. SQL Server additionally supports [abc] (any one of), [^abc] (none of) and [a-f] (range); in MySQL use REGEXP for these.

SYNTAX

%	zero or more characters	'bl%' -> bl, black, blue
_	exactly one character	'h_t' -> hot, hat
[]	any one char inside (SQL Server)	'h[oa]t' -> hot, hat
^	NOT inside brackets (SQL Server)	'h[^oa]t' -> hit
-	range (SQL Server)	'c[a-b]t' -> cat, cbt

Problem 2.19.1 Postal codes that are exactly 5 characters long.

SOLUTION (SQL)

```
SELECT CustomerName, PostalCode FROM Customers
WHERE PostalCode LIKE '_____';
```

RESULT

```
+-----+-----+
| CustomerName | PostalCode |
+-----+-----+
| Berlin Bazaar | 10117      |
| Munich Deli   | 80331      |
| New York Masala | 10016      |
| Texas Tandoor | 77002      |
+-----+-----+
4 rows in set
```

Problem 2.19.2 Customer names whose third letter is 'r'.

SOLUTION (SQL)

```
SELECT CustomerName FROM Customers
WHERE CustomerName LIKE '___r%';
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Curry Corner |
| Berlin Bazaar |
+-----+
2 rows in set
```

Problem 2.19.3 Customers whose name starts with B, D or L (MySQL: use REGEXP).

SOLUTION (SQL)

```
SELECT CustomerName FROM Customers
WHERE CustomerName REGEXP '[BDL]';
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Desi Mart    |
| London Spice House |
| Berlin Bazaar |
| Dubai Gourmet |
+-----+
4 rows in set
```

Tip: SQL Server equivalent: WHERE CustomerName LIKE '[BDL]%'

Problem 2.19.4 Customers whose name starts with a letter between A and F.

SOLUTION (SQL)

```
SELECT CustomerName FROM Customers
WHERE CustomerName REGEXP '^[A-F]';
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Fresh Basket  |
| Desi Mart     |
| Curry Corner  |
| Berlin Bazaar |
| Dubai Gourmet |
+-----+
5 rows in set
```

Tip: SQL Server equivalent: WHERE CustomerName LIKE '[A-F]%'

2.20 IN

IN is a shorthand for multiple OR conditions. It also accepts a subquery.

SYNTAX

```
SELECT columns FROM table_name WHERE column IN (value1, value2, ...);
SELECT columns FROM table_name WHERE column IN (SELECT ...);
```

Problem 2.20.1 Customers located in the UK, Germany or USA.

SOLUTION (SQL)

```
SELECT CustomerName, Country
FROM Customers
WHERE Country IN ('UK', 'Germany', 'USA');
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| London Spice House | UK      |
| Curry Corner      | UK      |
| Berlin Bazaar      | Germany |
| Munich Deli        | Germany |
| New York Masala     | USA     |
| Texas Tandoor      | USA     |
+-----+-----+
6 rows in set
```

Problem 2.20.2 Customers NOT in India or the UAE.

SOLUTION (SQL)

```
SELECT CustomerName, Country
FROM Customers
WHERE Country NOT IN ('India', 'UAE');
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| London Spice House | UK      |
| Curry Corner      | UK      |
| Berlin Bazaar     | Germany |
| Munich Deli       | Germany |
| New York Masala   | USA     |
| Texas Tandoor     | USA     |
+-----+-----+
6 rows in set
```

Problem 2.20.3 Customers who have placed at least one order (IN with subquery).

SOLUTION (SQL)

```
SELECT CustomerName
FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM Orders);
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Sharma General Store |
| Patel Supermart      |
| Mehta Kirana         |
| Fresh Basket        |
| Desi Mart           |
| London Spice House   |
| Curry Corner         |
| Berlin Bazaar       |
| Munich Deli         |
| Dubai Gourmet       |
| Sharjah Traders     |
| New York Masala     |
+-----+
12 rows in set
```

Problem 2.20.4 Customers who have never ordered.

SOLUTION (SQL)

```
SELECT CustomerName
FROM Customers
WHERE CustomerID NOT IN (SELECT CustomerID FROM Orders);
```

RESULT

```
+-----+
| CustomerName |
+-----+
| Texas Tandoor |
| Kolkata Sweets |
| Pune Pantry   |
+-----+
3 rows in set
```

2.21 BETWEEN

BETWEEN selects values within a range; both ends are included. Works with numbers, text and dates.

SYNTAX

```
SELECT columns FROM table_name WHERE column BETWEEN value1 AND value2;
```

Problem 2.21.1 Products priced between 100 and 400.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE Price BETWEEN 100 AND 400
ORDER BY Price;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Bhujia Namkeen | 110.00 |
| Mango Juice | 120.00 |
| Masala Chai | 250.00 |
| Filter Coffee | 320.00 |
+-----+-----+
4 rows in set
```

Problem 2.21.2 Products priced between 100 and 400 but NOT in category 1 or 2.

SOLUTION (SQL)

```
SELECT ProductName, CategoryID, Price
FROM Products
WHERE Price BETWEEN 100 AND 400
AND CategoryID NOT IN (1, 2);
```

RESULT

Empty set

Problem 2.21.3 Orders placed in the first quarter of 2026.

SOLUTION (SQL)

```
SELECT OrderID, OrderDate
FROM Orders
WHERE OrderDate BETWEEN '2026-01-01' AND '2026-03-31';
```

RESULT

```
+-----+-----+
| OrderID | OrderDate |
+-----+-----+
| 1001 | 2026-01-05 |
| 1002 | 2026-01-12 |
| 1003 | 2026-01-20 |
| 1004 | 2026-02-03 |
| 1005 | 2026-02-14 |
| 1006 | 2026-02-25 |
| 1007 | 2026-03-04 |
| 1008 | 2026-03-15 |
| 1009 | 2026-03-28 |
+-----+-----+
9 rows in set
```

Problem 2.21.4 Products whose names fall alphabetically between 'Banana Chips' and 'Ghee'.

SOLUTION (SQL)

```
SELECT ProductName
FROM Products
WHERE ProductName BETWEEN 'Banana Chips' AND 'Ghee'
ORDER BY ProductName;
```

RESULT

```
+-----+
| ProductName |
+-----+
| Banana Chips |
| Basmati Rice |
| Bhujia Namkeen |
| Filter Coffee |
| Garam Masala |
| Ghee |
+-----+
6 rows in set
```

2.22 Aliases (AS)

An alias gives a temporary name to a column or table for the duration of a query - making results readable and joins shorter.

SYNTAX

```
SELECT column_name AS alias_name FROM table_name;
SELECT column_name(s) FROM table_name AS alias_name;
```

Problem 2.22.1 Show CustomerID as ID and CustomerName as Customer.

SOLUTION (SQL)

```
SELECT CustomerID AS ID, CustomerName AS Customer
FROM Customers
LIMIT 5;
```

RESULT

```
+---+-----+
| ID | Customer |
+---+-----+
| 1 | Sharma General Store |
| 2 | Patel Supermart |
| 3 | Mehta Kirana |
| 4 | Fresh Basket |
| 5 | Desi Mart |
+---+-----+
5 rows in set
```

Problem 2.22.2 Use an alias that contains a space (wrap it in back-ticks or double quotes).

SOLUTION (SQL)

```
SELECT ProductName AS `Product Name`, Price AS `Price (INR)`
FROM Products
LIMIT 4;
```

RESULT

```
+-----+-----+
| Product Name | Price (INR) |
+-----+-----+
| Masala Chai | 250.00 |
| Filter Coffee | 320.00 |
| Mango Juice | 120.00 |
| Banana Chips | 90.00 |
+-----+-----+
4 rows in set
```

Problem 2.22.3 Combine several columns into one 'FullAddress' column.

SOLUTION (SQL)

```
SELECT CustomerName,  
       CONCAT(Address, ', ', City, ' - ', PostalCode, ', ', Country) AS FullAddress  
FROM Customers  
WHERE Country = 'India';
```

RESULT

CustomerName	FullAddress
Sharma General Store	12 MG Road, Ahmedabad - 380001, India
Patel Supermart	45 CG Road, Ahmedabad - 380009, India
Mehta Kirana	7 Station Road, Surat - 395003, India
Fresh Basket	21 Brigade Road, Bengaluru - 560025, India
Desi Mart	88 Linking Road, Mumbai - 400050, India
Kolkata Sweets	15 Park Street, Kolkata - 700016, India
Pune Pantry	9 FC Road, Pune - 411004, India

7 rows in set

Tip: SQL Server: Address + ', ' + City ... Oracle: Address || ', ' || City ...

Problem 2.22.4 Use table aliases (c, o) to shorten a query: orders of customer 'Desi Mart'.

SOLUTION (SQL)

```
SELECT o.OrderID, o.OrderDate, c.CustomerName  
FROM Customers AS c, Orders AS o  
WHERE c.CustomerName = 'Desi Mart'  
      AND c.CustomerID = o.CustomerID;
```

RESULT

OrderID	OrderDate	CustomerName
1005	2026-02-14	Desi Mart
1015	2026-06-11	Desi Mart

2 rows in set

2.23 JOINS - overview

A JOIN combines rows from two or more tables based on a related column. INNER JOIN = only matching rows; LEFT JOIN = all rows from the left table; RIGHT JOIN = all rows from the right table; FULL JOIN = all rows from both.

SYNTAX

```
SELECT columns  
FROM tableA  
[INNER | LEFT | RIGHT | FULL OUTER] JOIN tableB  
  ON tableA.column = tableB.column;
```

Problem 2.23.1 Show each order with the customer's name (Orders.CustomerID links to Customers.CustomerID).

SOLUTION (SQL)

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

RESULT

```
+-----+-----+-----+
| OrderID | CustomerName      | OrderDate |
+-----+-----+-----+
| 1001 | Sharma General Store | 2026-01-05 |
| 1002 | Patel Supermart     | 2026-01-12 |
| 1003 | London Spice House  | 2026-01-20 |
| 1004 | Fresh Basket        | 2026-02-03 |
| 1005 | Desi Mart           | 2026-02-14 |
| 1006 | Sharma General Store | 2026-02-25 |
| 1007 | Berlin Bazaar       | 2026-03-04 |
| 1008 | Dubai Gourmet       | 2026-03-15 |
| 1009 | Mehta Kirana        | 2026-03-28 |
| 1010 | New York Masala     | 2026-04-06 |
| 1011 | Patel Supermart     | 2026-04-18 |
| 1012 | Curry Corner        | 2026-05-02 |
| 1013 | Munich Deli         | 2026-05-20 |
| 1014 | Sharjah Traders     | 2026-06-01 |
| 1015 | Desi Mart           | 2026-06-11 |
+-----+-----+-----+
15 rows in set
```

2.24 INNER JOIN

INNER JOIN returns only rows that have matching values in both tables. JOIN on its own means INNER JOIN.

SYNTAX

```
SELECT columns FROM table1
INNER JOIN table2 ON table1.column = table2.column;
```

Problem 2.24.1 List every product with its category name.

SOLUTION (SQL)

```
SELECT p.ProductName, c.CategoryName
FROM Products p
INNER JOIN Categories c ON p.CategoryID = c.CategoryID;
```

RESULT

```
+-----+-----+
| ProductName | CategoryName |
+-----+-----+
| Masala Chai | Beverages    |
| Filter Coffee | Beverages    |
| Mango Juice | Beverages    |
| Banana Chips | Snacks       |
| Bhujia Namkeen | Snacks       |
| Paneer      | Dairy        |
| Ghee        | Dairy        |
| Basmati Rice | Grains       |
| Whole Wheat Atta | Grains       |
| Garam Masala | Spices       |
| Saffron     | Spices       |
| Kaju Katli  | Sweets       |
| Gulab Jamun Mix | Sweets       |
+-----+-----+
13 rows in set
```

Problem 2.24.2 Show orders with customer name AND shipper name (joining three tables).

SOLUTION (SQL)

```
SELECT o.OrderID, c.CustomerName, s.ShipperName
FROM ((Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID)
INNER JOIN Shippers s ON o.ShipperID = s.ShipperID);
```

RESULT

OrderID	CustomerName	ShipperName
1002	Patel Supermart	BlueDart Express
1004	Fresh Basket	BlueDart Express
1009	Mehta Kirana	BlueDart Express
1011	Patel Supermart	BlueDart Express
1001	Sharma General Store	Delhivery
1005	Desi Mart	Delhivery
1006	Sharma General Store	Delhivery
1014	Sharjah Traders	Delhivery
1015	Desi Mart	Delhivery
1003	London Spice House	DHL Global
1007	Berlin Bazaar	DHL Global
1008	Dubai Gourmet	DHL Global
1010	New York Masala	DHL Global
1012	Curry Corner	DHL Global
1013	Munich Deli	DHL Global

15 rows in set

Problem 2.24.3 Show every line of order 1003 with product name, quantity and line total.

SOLUTION (SQL)

```
SELECT od.OrderID, p.ProductName, od.Quantity, p.Price,
       od.Quantity * p.Price AS LineTotal
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
WHERE od.OrderID = 1003;
```

RESULT

OrderID	ProductName	Quantity	Price	LineTotal
1003	Garam Masala	50	93.50	4675.00
1003	Saffron	10	438.90	4389.00
1003	Masala Chai	25	250.00	6250.00

3 rows in set

2.25 LEFT JOIN

LEFT JOIN returns all rows from the left table, plus matching rows from the right table. Where there is no match, the right-side columns are NULL.

SYNTAX

```
SELECT columns FROM table1
LEFT JOIN table2 ON table1.column = table2.column;
```

Problem 2.25.1 List ALL customers and their order IDs - including customers with no orders.

SOLUTION (SQL)

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
ORDER BY c.CustomerName;
```

RESULT

CustomerName	OrderID
Berlin Bazaar	1007
Curry Corner	1012
Desi Mart	1005
Desi Mart	1015
Dubai Gourmet	1008
Fresh Basket	1004
Kolkata Sweets	NULL
London Spice House	1003
Mehta Kirana	1009
Munich Deli	1013
New York Masala	1010
Patel Supermart	1002
Patel Supermart	1011
Pune Pantry	NULL
Sharjah Traders	1014
Sharma General Store	1001
Sharma General Store	1006
Texas Tandoor	NULL

18 rows in set

Problem 2.25.2 Find only the customers who have never placed an order.

SOLUTION (SQL)

```
SELECT c.CustomerName, c.City
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

RESULT

CustomerName	City
Texas Tandoor	Houston
Kolkata Sweets	Kolkata
Pune Pantry	Pune

3 rows in set

2.26 RIGHT JOIN

RIGHT JOIN returns all rows from the right table, plus matching rows from the left table (NULL where no match).

SYNTAX

```
SELECT columns FROM table1
RIGHT JOIN table2 ON table1.column = table2.column;
```

Problem 2.26.1 List ALL employees and the orders they handled - including employees with no orders.

SOLUTION (SQL)

```
SELECT o.OrderID, e.FirstName, e.LastName
FROM Orders o
RIGHT JOIN Employees e ON o.EmployeeID = e.EmployeeID
ORDER BY e.EmployeeID, o.OrderID;
```

RESULT

```
+-----+-----+-----+
| OrderID | FirstName | LastName |
+-----+-----+-----+
| 1013 | Arjun | Mehta |
| 1004 | Kavya | Iyer |
| 1011 | Kavya | Iyer |
| 1001 | Rohan | Desai |
| 1002 | Rohan | Desai |
| 1006 | Rohan | Desai |
| 1009 | Rohan | Desai |
| 1003 | Sara | Thomas |
| 1007 | Sara | Thomas |
| 1008 | Sara | Thomas |
| 1010 | Sara | Thomas |
| 1012 | Sara | Thomas |
| 1005 | Vikram | Singh |
| 1014 | Vikram | Singh |
| 1015 | Vikram | Singh |
| NULL | Neha | Joshi |
+-----+-----+-----+
16 rows in set
```

Problem 2.26.2 List every supplier and its products, keeping suppliers without products.

SOLUTION (SQL)

```
SELECT s.SupplierName, p.ProductName
FROM Products p
RIGHT JOIN Suppliers s ON p.SupplierID = s.SupplierID
ORDER BY s.SupplierName;
```

RESULT

```
+-----+-----+
| SupplierName | ProductName |
+-----+-----+
| Amul Traders | Bhujia Namkeen |
| Amul Traders | Paneer |
| Amul Traders | Ghee |
| Amul Traders | Whole Wheat Atta |
| Bavaria Imports | Kaju Katli |
| Gulf Fresh LLC | Basmati Rice |
| Gulf Fresh LLC | Mango Juice |
| Himalaya Foods | Banana Chips |
| Himalaya Foods | Masala Chai |
| Himalaya Foods | Filter Coffee |
| Kerala Coconut Co | NULL |
| Spice Route Ltd | Garam Masala |
| Spice Route Ltd | Saffron |
+-----+-----+
13 rows in set
```

2.27 FULL OUTER JOIN

FULL OUTER JOIN returns all rows from both tables, matched where possible. SQL Server, PostgreSQL and Oracle support it directly; MySQL does NOT, so we emulate it with LEFT JOIN UNION RIGHT JOIN.

SYNTAX

```
-- SQL Server / PostgreSQL / Oracle
SELECT columns FROM table1
FULL OUTER JOIN table2 ON table1.column = table2.column;
```

Problem 2.27.1 Show all products and all suppliers: products without a supplier AND suppliers without products must both appear.

SOLUTION (SQL)

```
SELECT p.ProductName, s.SupplierName
FROM Products p
LEFT JOIN Suppliers s ON p.SupplierID = s.SupplierID
UNION
SELECT p.ProductName, s.SupplierName
FROM Products p
RIGHT JOIN Suppliers s ON p.SupplierID = s.SupplierID;
```

RESULT

```
+-----+-----+
| ProductName | SupplierName |
+-----+-----+
| Masala Chai | Himalaya Foods |
| Filter Coffee | Himalaya Foods |
| Mango Juice | Gulf Fresh LLC |
| Banana Chips | Himalaya Foods |
| Bhujia Namkeen | Amul Traders |
| Paneer | Amul Traders |
| Ghee | Amul Traders |
| Basmati Rice | Gulf Fresh LLC |
| Whole Wheat Atta | Amul Traders |
| Garam Masala | Spice Route Ltd |
| Saffron | Spice Route Ltd |
| Kaju Katli | Bavaria Imports |
| Gulab Jamun Mix | NULL |
| NULL | Kerala Coconut Co |
+-----+-----+
14 rows in set
```

Tip: Gulab Jamun Mix (no supplier) and Kerala Coconut Co (no products) both appear with NULL on the other side.

2.28 SELF JOIN

A self join joins a table with itself - useful for hierarchies (employee -> manager) or comparing rows of the same table. Table aliases are required.

SYNTAX

```
SELECT columns FROM table1 T1, table1 T2 WHERE condition;
```

Problem 2.28.1 Show each employee with the name of their manager.

SOLUTION (SQL)

```
SELECT CONCAT(e.FirstName, ' ', e.LastName) AS Employee,
       CONCAT(m.FirstName, ' ', m.LastName) AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

RESULT

```
+-----+-----+
| Employee | Manager |
+-----+-----+
| Arjun Mehta | NULL |
| Kavya Iyer | Arjun Mehta |
| Rohan Desai | Arjun Mehta |
| Sara Thomas | Kavya Iyer |
| Vikram Singh | Kavya Iyer |
| Neha Joshi | Rohan Desai |
+-----+-----+
6 rows in set
```

Problem 2.28.2 Find pairs of customers located in the same city.

SOLUTION (SQL)

```
SELECT A.CustomerName AS Customer1, B.CustomerName AS Customer2, A.City
FROM Customers A, Customers B
WHERE A.CustomerID < B.CustomerID
      AND A.City = B.City;
```

RESULT

```
+-----+-----+-----+
| Customer1 | Customer2 | City |
+-----+-----+-----+
| Sharma General Store | Patel Supermart | Ahmedabad |
+-----+-----+-----+
1 row in set
```

2.29 UNION

UNION combines the results of two or more SELECTs and removes duplicates. Each SELECT must have the same number of columns with similar data types.

SYNTAX

```
SELECT column(s) FROM table1
UNION
SELECT column(s) FROM table2;
```

Problem 2.29.1 List every distinct city where we have a customer or a supplier.

SOLUTION (SQL)

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers
ORDER BY City;
```

RESULT

```
+-----+
| City |
+-----+
| Ahmedabad |
| Anand |
| Bengaluru |
| Berlin |
| Dehradun |
| Dubai |
| Houston |
| Kochi |
| Kolkata |
| London |
| Manchester |
| Mumbai |
| Munich |
| New York |
| Pune |
| Sharjah |
| Surat |
+-----+
17 rows in set
```

Problem 2.29.2 Indian cities only, from both tables.

SOLUTION (SQL)

```
SELECT City, Country FROM Customers WHERE Country = 'India'
UNION
SELECT City, Country FROM Suppliers WHERE Country = 'India'
ORDER BY City;
```

RESULT

```
+-----+-----+
| City   | Country |
+-----+-----+
| Ahmedabad | India |
| Anand    | India |
| Bengaluru | India |
| Dehradun | India |
| Kochi    | India |
| Kolkata  | India |
| Mumbai   | India |
| Pune     | India |
| Surat    | India |
+-----+-----+
9 rows in set
```

Problem 2.29.3 Build a contact list of customers and suppliers, labelled with their type.

SOLUTION (SQL)

```
SELECT 'Customer' AS Type, ContactName, City, Country FROM Customers WHERE Country = 'UK'
UNION
SELECT 'Supplier', ContactName, City, Country FROM Suppliers WHERE Country = 'UK';
```

RESULT

```
+-----+-----+-----+-----+
| Type   | ContactName | City       | Country |
+-----+-----+-----+-----+
| Customer | James Brown | London     | UK      |
| Customer | Sophie Clark | Manchester | UK      |
| Supplier | Oliver Smith | London     | UK      |
+-----+-----+-----+-----+
3 rows in set
```

2.30 UNION ALL

UNION ALL works like UNION but keeps duplicate rows (and is faster).

SYNTAX

```
SELECT column(s) FROM table1
UNION ALL
SELECT column(s) FROM table2;
```

Problem 2.30.1 List the countries of all customers and suppliers, keeping duplicates.

SOLUTION (SQL)

```
SELECT Country FROM Customers
UNION ALL
SELECT Country FROM Suppliers
ORDER BY Country;
```

RESULT

```
+-----+
| Country |
+-----+
| Germany |
| Germany |
| Germany |
| India   |
| India   |
| India   |
| India   |
| India   |
| India   |
| India   |
| India   |
| India   |
| India   |
| India   |
| India   |
| UAE     |
| UAE     |
| UAE     |
| UK       |
| UK       |
| UK       |
| USA     |
| USA     |
+-----+
21 rows in set
```

2.31 GROUP BY

GROUP BY puts rows with the same values into groups so that aggregate functions can be calculated per group.

SYNTAX

```
SELECT column, AGG_FUNCTION(column)
FROM table_name
WHERE condition
GROUP BY column
ORDER BY ...;
```

Problem 2.31.1 How many customers are there in each country? Show the biggest country first.

SOLUTION (SQL)

```
SELECT Country, COUNT(CustomerID) AS Customers
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

RESULT

```
+-----+-----+
| Country | Customers |
+-----+-----+
| India   |          7 |
| USA     |          2 |
| UAE     |          2 |
| Germany |          2 |
| UK      |          2 |
+-----+-----+
5 rows in set
```

Problem 2.31.2 How many orders did each shipper deliver?

SOLUTION (SQL)

```
SELECT s.ShipperName, COUNT(o.OrderID) AS NumberOfOrders
FROM Orders o
LEFT JOIN Shippers s ON o.ShipperID = s.ShipperID
GROUP BY s.ShipperName;
```

RESULT

```
+-----+-----+
| ShipperName | NumberOfOrders |
+-----+-----+
| BlueDart Express | 4 |
| Delhivery | 5 |
| DHL Global | 6 |
+-----+-----+
3 rows in set
```

Problem 2.31.3 What is the revenue for each category?

SOLUTION (SQL)

```
SELECT c.CategoryName, SUM(od.Quantity * p.Price) AS Revenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryName
ORDER BY Revenue DESC;
```

RESULT

```
+-----+-----+
| CategoryName | Revenue |
+-----+-----+
| Grains | 63660.00 |
| Beverages | 45330.00 |
| Spices | 19912.20 |
| Sweets | 18900.00 |
| Snacks | 16800.00 |
| Dairy | 12075.00 |
+-----+-----+
6 rows in set
```

2.32 HAVING

WHERE cannot use aggregate functions, so HAVING filters groups AFTER GROUP BY.

SYNTAX

```
SELECT column, AGG(column)
FROM table_name
WHERE condition
GROUP BY column
HAVING AGG_condition
ORDER BY ...;
```

Problem 2.32.1 Which countries have more than 2 customers?

SOLUTION (SQL)

```
SELECT Country, COUNT(CustomerID) AS Customers
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 2;
```

RESULT

```
+-----+-----+
| Country | Customers |
+-----+-----+
| India | 7 |
+-----+-----+
1 row in set
```

Problem 2.32.2 Which employees handled more than 3 orders?

SOLUTION (SQL)

```
SELECT e.FirstName, e.LastName, COUNT(o.OrderID) AS NumberOfOrders
FROM Orders o
INNER JOIN Employees e ON o.EmployeeID = e.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
HAVING COUNT(o.OrderID) > 3;
```

RESULT

```
+-----+-----+-----+
| FirstName | LastName | NumberOfOrders |
+-----+-----+-----+
| Rohan     | Desai    | 4              |
| Sara      | Thomas   | 5              |
+-----+-----+-----+
2 rows in set
```

Problem 2.32.3 Which categories earned more than 20,000 in revenue?

SOLUTION (SQL)

```
SELECT c.CategoryName, SUM(od.Quantity * p.Price) AS Revenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryName
HAVING Revenue > 20000;
```

RESULT

```
+-----+-----+
| CategoryName | Revenue |
+-----+-----+
| Beverages    | 45330.00 |
| Grains       | 63660.00 |
+-----+-----+
2 rows in set
```

2.33 EXISTS

EXISTS tests whether a subquery returns any row. It returns TRUE as soon as one row is found.

SYNTAX

```
SELECT columns FROM table_name
WHERE EXISTS (SELECT column FROM table_name WHERE condition);
```

Problem 2.33.1 List suppliers that supply at least one product cheaper than 100.

SOLUTION (SQL)

```
SELECT SupplierName
FROM Suppliers s
WHERE EXISTS (SELECT ProductName FROM Products p
              WHERE p.SupplierID = s.SupplierID AND p.Price < 100);
```

RESULT

```
+-----+
| SupplierName |
+-----+
| Amul Traders |
| Himalaya Foods |
| Spice Route Ltd |
+-----+
3 rows in set
```

Problem 2.33.2 List suppliers that have NO products at all.

SOLUTION (SQL)

```
SELECT SupplierName
FROM Suppliers s
WHERE NOT EXISTS (SELECT 1 FROM Products p WHERE p.SupplierID = s.SupplierID);
```

RESULT

```
+-----+
| SupplierName |
+-----+
| Kerala Coconut Co |
+-----+
1 row in set
```

2.34 ANY

ANY returns TRUE if the comparison is true for at least one value returned by the subquery.

SYNTAX

```
SELECT columns FROM table_name
WHERE column operator ANY (SELECT column FROM table_name WHERE condition);
```

Problem 2.34.1 Which products were ordered in a quantity of exactly 10 in any order line?

SOLUTION (SQL)

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY (SELECT ProductID FROM OrderDetails WHERE Quantity = 10);
```

RESULT

```
+-----+
| ProductName |
+-----+
| Masala Chai |
| Paneer      |
| Whole Wheat Atta |
| Saffron     |
| Kaju Katli  |
+-----+
5 rows in set
```

Problem 2.34.2 Which products were ordered in quantities greater than 40 at least once?

SOLUTION (SQL)

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY (SELECT ProductID FROM OrderDetails WHERE Quantity > 40);
```

RESULT

```
+-----+
| ProductName |
+-----+
| Mango Juice |
| Garam Masala |
+-----+
2 rows in set
```

2.35 ALL

ALL returns TRUE only if the comparison is true for every value returned by the subquery.

SYNTAX

```
SELECT columns FROM table_name
WHERE column operator ALL (SELECT column FROM table_name WHERE condition);
```

Problem 2.35.1 Which products are more expensive than ALL snacks (CategoryID 2)?

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE Price > ALL (SELECT Price FROM Products WHERE CategoryID = 2)
ORDER BY Price;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Mango Juice | 120.00 |
| Masala Chai | 250.00 |
| Filter Coffee | 320.00 |
| Saffron | 438.90 |
| Whole Wheat Atta | 460.00 |
| Kaju Katli | 540.00 |
| Ghee | 650.00 |
| Basmati Rice | 780.00 |
+-----+-----+
8 rows in set
```

Problem 2.35.2 Find the most expensive product using >= ALL.

SOLUTION (SQL)

```
SELECT ProductName, Price
FROM Products
WHERE Price >= ALL (SELECT Price FROM Products);
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Basmati Rice | 780.00 |
+-----+-----+
1 row in set
```

Problem 2.35.3 ALL can also be used with SELECT: select all rows including duplicates (the default).

SOLUTION (SQL)

```
SELECT ALL Country FROM Suppliers;
```

RESULT

```
+-----+
| Country |
+-----+
| India |
| India |
| UK |
| Germany |
| UAE |
| India |
+-----+
6 rows in set
```

2.36 SELECT INTO

SELECT INTO copies data into a NEW table (SQL Server / MS Access / PostgreSQL). MySQL uses CREATE TABLE ... AS SELECT instead. In MySQL, SELECT ... INTO stores a value into a variable.

SYNTAX

```
-- SQL Server
SELECT * INTO newtable FROM oldtable WHERE condition;
-- MySQL equivalent
CREATE TABLE newtable AS SELECT * FROM oldtable WHERE condition;
```

Problem 2.36.1 Create a full backup copy of the Customers table.

SOLUTION (SQL)

```
CREATE TABLE CustomersBackup AS
SELECT * FROM Customers;

SELECT COUNT(*) AS RowsCopied FROM CustomersBackup;
```

RESULT

```
Query OK, 15 rows affected
Records: 15 Duplicates: 0 Warnings: 0

+-----+
| RowsCopied |
+-----+
|          15 |
+-----+
1 row in set
```

Tip: SQL Server: `SELECT * INTO CustomersBackup FROM Customers;`

Problem 2.36.2 Copy only Indian customers (and only a few columns) into a new table.

SOLUTION (SQL)

```
CREATE TABLE IndiaCustomers AS
SELECT CustomerName, ContactName, City
FROM Customers
WHERE Country = 'India';

SELECT * FROM IndiaCustomers;
```

RESULT

```
Query OK, 7 rows affected
Records: 7 Duplicates: 0 Warnings: 0

+-----+-----+-----+
| CustomerName | ContactName | City |
+-----+-----+-----+
| Sharma General Store | Rakesh Sharma | Ahmedabad |
| Patel Supermart | Nisha Patel | Ahmedabad |
| Mehta Kirana | Anil Mehta | Surat |
| Fresh Basket | Priya Nair | Bengaluru |
| Desi Mart | Imran Khan | Mumbai |
| Kolkata Sweets | Sourav Das | Kolkata |
| Pune Pantry | Rahul Joshi | Pune |
+-----+-----+-----+
7 rows in set
```

Problem 2.36.3 Create an EMPTY table with the same structure as Products (copy no rows).

SOLUTION (SQL)

```
CREATE TABLE ProductsTemplate AS
SELECT * FROM Products WHERE 1 = 0;

SELECT COUNT(*) AS RowsInTemplate FROM ProductsTemplate;
```

RESULT

```
Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

+-----+
| RowsInTemplate |
+-----+
|          0 |
+-----+
1 row in set
```

Problem 2.36.4 MySQL variable: store the highest price into @maxPrice and use it.

SOLUTION (SQL)

```
SELECT MAX(Price) INTO @maxPrice FROM Products;

SELECT ProductName, Price FROM Products WHERE Price = @maxPrice;
```

RESULT

Query OK, 1 row affected

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Basmati Rice | 780.00 |
+-----+-----+
1 row in set
```

2.37 INSERT INTO SELECT

INSERT INTO SELECT copies rows from one table into an EXISTING table. Column count and types must match.

SYNTAX

```
INSERT INTO table2 (column1, column2, ...)
SELECT column1, column2, ... FROM table1 WHERE condition;
```

Problem 2.37.1 Create a single contact directory table and fill it with both customers and suppliers.

SOLUTION (SQL)

```
CREATE TABLE AllContacts (
    Name    VARCHAR(100),
    City    VARCHAR(50),
    Country VARCHAR(50),
    Type    VARCHAR(10)
);

INSERT INTO AllContacts (Name, City, Country, Type)
SELECT CustomerName, City, Country, 'Customer' FROM Customers;

INSERT INTO AllContacts (Name, City, Country, Type)
SELECT SupplierName, City, Country, 'Supplier' FROM Suppliers;

SELECT Type, COUNT(*) AS Total FROM AllContacts GROUP BY Type;
```

RESULT

Query OK, 0 rows affected

Query OK, 15 rows affected

Records: 15 Duplicates: 0 Warnings: 0

Query OK, 6 rows affected

Records: 6 Duplicates: 0 Warnings: 0

```
+-----+-----+
| Type    | Total |
+-----+-----+
| Customer |    15 |
| Supplier |     6 |
+-----+-----+
2 rows in set
```

Problem 2.37.2 Copy only German suppliers into the IndiaCustomers-style table 'ForeignPartners'.

SOLUTION (SQL)

```
CREATE TABLE ForeignPartners (Name VARCHAR(100), City VARCHAR(50));

INSERT INTO ForeignPartners (Name, City)
SELECT SupplierName, City FROM Suppliers WHERE Country = 'Germany';

SELECT * FROM ForeignPartners;
```

RESULT

```
Query OK, 0 rows affected

Query OK, 1 row affected
Records: 1 Duplicates: 0 Warnings: 0

+-----+-----+
| Name          | City   |
+-----+-----+
| Bavaria Imports | Munich |
+-----+-----+
1 row in set
```

2.38 CASE

CASE goes through conditions and returns a value when the first condition is met (like if-then-else). If none match, ELSE is returned (or NULL if there is no ELSE).

SYNTAX

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE result
END
```

Problem 2.38.1 Label every product as Budget, Mid-range or Premium based on its price.

SOLUTION (SQL)

```
SELECT ProductName, Price,
       CASE
         WHEN Price < 100 THEN 'Budget'
         WHEN Price <= 400 THEN 'Mid-range'
         ELSE 'Premium'
       END AS PriceBand
FROM Products
ORDER BY Price;
```

RESULT

```
+-----+-----+-----+
| ProductName | Price | PriceBand |
+-----+-----+-----+
| Gulab Jamun Mix | 75.00 | Budget |
| Banana Chips | 90.00 | Budget |
| Garam Masala | 93.50 | Budget |
| Paneer | 95.00 | Budget |
| Bhujia Namkeen | 110.00 | Mid-range |
| Mango Juice | 120.00 | Mid-range |
| Masala Chai | 250.00 | Mid-range |
| Filter Coffee | 320.00 | Mid-range |
| Saffron | 438.90 | Premium |
| Whole Wheat Atta | 460.00 | Premium |
| Kaju Katli | 540.00 | Premium |
| Ghee | 650.00 | Premium |
| Basmati Rice | 780.00 | Premium |
+-----+-----+-----+
13 rows in set
```

Problem 2.38.2 Describe each order line's quantity in words.

SOLUTION (SQL)

```
SELECT OrderID, Quantity,
       CASE
         WHEN Quantity > 30 THEN 'Bulk order'
         WHEN Quantity = 30 THEN 'Exactly 30'
         ELSE 'Regular order'
       END AS QuantityText
FROM OrderDetails
WHERE OrderID IN (1001, 1003, 1010);
```

RESULT

```
+-----+-----+-----+
| OrderID | Quantity | QuantityText |
+-----+-----+-----+
| 1001 | 10 | Regular order |
| 1001 | 20 | Regular order |
| 1001 | 15 | Regular order |
| 1003 | 50 | Bulk order |
| 1003 | 10 | Regular order |
| 1003 | 25 | Regular order |
| 1010 | 30 | Exactly 30 |
| 1010 | 15 | Regular order |
| 1010 | 8 | Regular order |
+-----+-----+-----+
9 rows in set
```

Problem 2.38.3 Sort customers by City, but if City is NULL sort by Country instead (CASE in ORDER BY).

SOLUTION (SQL)

```
SELECT CustomerName, City, Country
FROM Customers
ORDER BY
  (CASE WHEN City IS NULL THEN Country ELSE City END)
LIMIT 6;
```

RESULT

```
+-----+-----+-----+
| CustomerName | City | Country |
+-----+-----+-----+
| Sharma General Store | Ahmedabad | India |
| Patel Supermart | Ahmedabad | India |
| Fresh Basket | Bengaluru | India |
| Berlin Bazaar | Berlin | Germany |
| Dubai Gourmet | Dubai | UAE |
| Texas Tandoor | Houston | USA |
+-----+-----+-----+
6 rows in set
```

2.39 NULL Functions - IFNULL(), COALESCE(), NULLIF()

When a value can be NULL, calculations and display break. These functions replace NULL with a fallback. MySQL: IFNULL(), COALESCE(). SQL Server: ISNULL(), COALESCE(). MS Access: IsNull()/IIF. Oracle: NVL(), COALESCE().

SYNTAX

```
IFNULL(expression, alt_value)      -- MySQL
COALESCE(val1, val2, ..., valN)    -- all databases: first non-NULL
NULLIF(expr1, expr2)               -- NULL if equal, else expr1
```

Problem 2.39.1 Show postal codes, but print 'N/A' where the postal code is missing.

SOLUTION (SQL)

```
SELECT CustomerName, IFNULL(PostalCode, 'N/A') AS PostalCode
FROM Customers
WHERE Country IN ('UAE', 'Germany');
```

RESULT

```
+-----+-----+
| CustomerName | PostalCode |
+-----+-----+
| Berlin Bazaar | 10117      |
| Munich Deli   | 80331      |
| Dubai Gourmet | N/A        |
| Sharjah Traders | N/A        |
+-----+-----+
4 rows in set
```

Tip: SQL Server: ISNULL(PostalCode, 'N/A') Oracle: NVL(PostalCode, 'N/A')

Problem 2.39.2 Show a 'best available' contact: ContactName, else City, else 'Unknown'.

SOLUTION (SQL)

```
SELECT CustomerName,
       COALESCE(ContactName, City, 'Unknown') AS ReachVia
FROM Customers
WHERE CustomerName IN ('Pune Pantry', 'Desi Mart');
```

RESULT

```
+-----+-----+
| CustomerName | ReachVia   |
+-----+-----+
| Desi Mart    | Imran Khan |
| Pune Pantry  | Rahul Joshi |
+-----+-----+
2 rows in set
```

Problem 2.39.3 Show products whose SupplierID is NULL as 'No supplier' using COALESCE with a join.

SOLUTION (SQL)

```
SELECT p.ProductName, COALESCE(s.SupplierName, 'No supplier') AS Supplier
FROM Products p
LEFT JOIN Suppliers s ON p.SupplierID = s.SupplierID
WHERE p.CategoryID = 6;
```

RESULT

```
+-----+-----+
| ProductName | Supplier   |
+-----+-----+
| Kaju Katli  | Bavaria Imports |
| Gulab Jamun Mix | No supplier  |
+-----+-----+
2 rows in set
```

Problem 2.39.4 Use NULLIF to avoid 'division by zero' (returns NULL instead of an error).

SOLUTION (SQL)

```
SELECT 100 / NULLIF(0, 0) AS SafeDivision,
       100 / NULLIF(4, 0) AS NormalDivision;
```

RESULT

```
+-----+-----+
| SafeDivision | NormalDivision |
+-----+-----+
| NULL        | 25.0000        |
+-----+-----+
1 row in set
```

2.40 Stored Procedures

A stored procedure is saved SQL code you can call again and again, optionally with parameters. In the MySQL command line / Workbench you change the DELIMITER so the ; inside the body does not end the CREATE statement. SQL Server uses CREATE PROCEDURE ... AS ... GO and EXEC to run it.

SYNTAX

```
DELIMITER //
CREATE PROCEDURE procedure_name (IN param1 TYPE, ...)
BEGIN
    sql_statements;
END //
DELIMITER ;

CALL procedure_name(values);           -- MySQL
EXEC procedure_name @param = value;    -- SQL Server
```

Problem 2.40.1 Create a procedure that returns all customers and call it.

SOLUTION (SQL)

```
DELIMITER //
CREATE PROCEDURE SelectAllCustomers()
BEGIN
    SELECT CustomerID, CustomerName, Country FROM Customers;
END //
DELIMITER ;

CALL SelectAllCustomers();
```

RESULT

Query OK, 0 rows affected

CustomerID	CustomerName	Country
1	Sharma General Store	India
2	Patel Supermart	India
3	Mehta Kirana	India
4	Fresh Basket	India
5	Desi Mart	India
6	London Spice House	UK
7	Curry Corner	UK
8	Berlin Bazaar	Germany
9	Munich Deli	Germany
10	Dubai Gourmet	UAE
11	Sharjah Traders	UAE
12	New York Masala	USA
13	Texas Tandoor	USA
14	Kolkata Sweets	India
16	Pune Pantry	India

15 rows in set

Query OK, 0 rows affected

Problem 2.40.2 Create a procedure with one parameter: customers of a given country.

SOLUTION (SQL)

```
DELIMITER //
CREATE PROCEDURE GetCustomersByCountry(IN p_country VARCHAR(50))
BEGIN
    SELECT CustomerName, City FROM Customers WHERE Country = p_country;
END //
DELIMITER ;

CALL GetCustomersByCountry('Germany');
```

RESULT

Query OK, 0 rows affected

```
+-----+-----+
| CustomerName | City |
+-----+-----+
| Berlin Bazaar | Berlin |
| Munich Deli   | Munich |
+-----+-----+
2 rows in set
```

Query OK, 0 rows affected

Problem 2.40.3 Create a procedure with two parameters: customers of a country AND city.

SOLUTION (SQL)

```
DELIMITER //
CREATE PROCEDURE GetCustomersByCity(IN p_country VARCHAR(50), IN p_city VARCHAR(50))
BEGIN
    SELECT CustomerName, ContactName FROM Customers
    WHERE Country = p_country AND City = p_city;
END //
DELIMITER ;

CALL GetCustomersByCity('India', 'Ahmedabad');
```

RESULT

Query OK, 0 rows affected

```
+-----+-----+
| CustomerName | ContactName |
+-----+-----+
| Sharma General Store | Rakesh Sharma |
| Patel Supermart     | Nisha Patel   |
+-----+-----+
2 rows in set
```

Query OK, 0 rows affected

Problem 2.40.4 Create a procedure with an OUT parameter that returns the number of products in a category.

SOLUTION (SQL)

```
DELIMITER //
CREATE PROCEDURE CountProducts(IN p_cat INT, OUT p_total INT)
BEGIN
    SELECT COUNT(*) INTO p_total FROM Products WHERE CategoryID = p_cat;
END //
DELIMITER ;

CALL CountProducts(1, @total);
SELECT @total AS BeverageProducts;
```

RESULT

```
Query OK, 0 rows affected

Query OK, 1 row affected

+-----+
| BeverageProducts |
+-----+
|                3 |
+-----+
1 row in set
```

Problem 2.40.5 SQL Server version of the same idea (reference only).

SOLUTION (SQL)

```
CREATE PROCEDURE SelectAllCustomers @City nvarchar(30)
AS
SELECT * FROM Customers WHERE City = @City
GO;

EXEC SelectAllCustomers @City = 'London';
```

NOTE

(SQL Server T-SQL syntax - not executed on MySQL.)

2.41 Comments

Comments explain SQL and are ignored by the database. -- is a single-line comment, /* ... */ spans multiple lines, and MySQL also accepts # for single-line comments.

Problem 2.41.1 Use a single-line comment to explain a query.

SOLUTION (SQL)

```
-- Select all Indian customers
SELECT CustomerName FROM Customers WHERE Country = 'India'; -- end-of-line comment works too
```

RESULT

```
+-----+
| CustomerName      |
+-----+
| Sharma General Store |
| Patel Supermart    |
| Mehta Kirana       |
| Fresh Basket       |
| Desi Mart          |
| Kolkata Sweets     |
| Pune Pantry        |
+-----+
7 rows in set
```

Problem 2.41.2 Use a multi-line comment and a comment in the middle of a statement.

SOLUTION (SQL)

```
/* This query lists products
   from the Spices category */
SELECT ProductName, /* Unit, */ Price
FROM Products
WHERE CategoryID = 5;
```

RESULT

```
+-----+-----+
| ProductName | Price |
+-----+-----+
| Garam Masala | 93.50 |
| Saffron      | 438.90 |
+-----+-----+
2 rows in set
```

Problem 2.41.3 Temporarily disable a condition with a comment while debugging.

SOLUTION (SQL)

```
SELECT CustomerName, Country FROM Customers
WHERE Country = 'UK'
# AND City = 'London'
;
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| London Spice House | UK |
| Curry Corner      | UK |
+-----+-----+
2 rows in set
```

2.42 Operators

Operators used inside expressions and WHERE clauses.

SYNTAX

```
Arithmetic : + - * / % (MySQL also DIV = integer division)
Comparison : = > < >= <= <> (!=)
Compound   : += -= *= /= (SQL Server only)
Bitwise    : & | ^
Logical    : ALL AND ANY BETWEEN EXISTS IN LIKE NOT OR SOME
```

Problem 2.42.1 Try the arithmetic operators.

SOLUTION (SQL)

```
SELECT 30 + 20 AS AddOp, 30 - 20 AS SubOp, 30 * 20 AS MulOp,
        30 / 20 AS DivOp, 17 % 5 AS ModOp, 17 DIV 5 AS IntDiv;
```

RESULT

```
+-----+-----+-----+-----+-----+-----+
| AddOp | SubOp | MulOp | DivOp | ModOp | IntDiv |
+-----+-----+-----+-----+-----+-----+
| 50    | 10    | 600   | 1.5000 | 2     | 3      |
+-----+-----+-----+-----+-----+-----+
1 row in set
```

Problem 2.42.2 Show the price of each spice product with 18% GST added (arithmetic on columns).

SOLUTION (SQL)

```
SELECT ProductName, Price, ROUND(Price * 1.18, 2) AS PriceWithGST
FROM Products
WHERE CategoryID = 5;
```

RESULT

```
+-----+-----+-----+
| ProductName | Price | PriceWithGST |
+-----+-----+-----+
| Garam Masala | 93.50 | 110.33 |
| Saffron      | 438.90 | 517.90 |
+-----+-----+-----+
2 rows in set
```

Problem 2.42.3 Try bitwise operators.

SOLUTION (SQL)

```
SELECT 5 & 1 AS BitAnd, 5 | 2 AS BitOr, 5 ^ 1 AS BitXor;
```

RESULT

```
+-----+-----+-----+
| BitAnd | BitOr | BitXor |
+-----+-----+-----+
| 1      | 7     | 4      |
+-----+-----+-----+
1 row in set
```

Problem 2.42.4 SOME is a synonym of ANY: products ordered in quantity 50 or more.

SOLUTION (SQL)

```
SELECT ProductName FROM Products
WHERE ProductID = SOME (SELECT ProductID FROM OrderDetails WHERE Quantity >= 50);
```

RESULT

```
+-----+
| ProductName |
+-----+
| Mango Juice |
| Garam Masala |
+-----+
2 rows in set
```

SQL Database - Structure, Constraints & Security

CREATE/DROP/BACKUP DB, tables, constraints, indexes, dates, views, injection, prepared statements

3.1 CREATE DATABASE

Creates a new SQL database. You need admin/create privileges.

SYNTAX

```
CREATE DATABASE databasename;
```

Problem 3.1.1 Create a practice database named TestDB and check that it exists.

SOLUTION (SQL)

```
CREATE DATABASE TestDB;

SHOW DATABASES LIKE 'TestDB';
```

RESULT

Query OK, 1 row affected

```
+-----+
| Database (TestDB) |
+-----+
| TestDB           |
+-----+
1 row in set
```

Problem 3.1.2 Create a database only if it does not already exist (no error on re-run).

SOLUTION (SQL)

```
CREATE DATABASE IF NOT EXISTS TestDB;

SHOW DATABASES LIKE 'TestDB';
```

RESULT

Query OK, 0 rows affected, 1 warning

```
+-----+
| Database (TestDB) |
+-----+
| TestDB           |
+-----+
1 row in set
```

3.2 DROP DATABASE

Deletes a database and EVERYTHING inside it. There is no undo - take a backup first.

SYNTAX

```
DROP DATABASE databasename;
```

Problem 3.2.1 Delete the TestDB practice database.

SOLUTION (SQL)

```
DROP DATABASE TestDB;

SHOW DATABASES LIKE 'TestDB';
```

RESULT

Query OK, 0 rows affected

Empty set

Problem 3.2.2 Drop a database only if it exists.

SOLUTION (SQL)

```
DROP DATABASE IF EXISTS TestDB;
```

RESULT

```
Query OK, 0 rows affected, 1 warning
```

3.3 BACKUP DATABASE

SQL Server has a BACKUP DATABASE statement. In MySQL, backups are taken with the mysqldump command-line tool (run in your terminal, not inside MySQL).

SYNTAX

```
-- SQL Server
BACKUP DATABASE databasename TO DISK = 'filepath';
BACKUP DATABASE databasename TO DISK = 'filepath' WITH DIFFERENTIAL;
```

Problem 3.3.1 MySQL: back up the whole ShopDB database to a file (terminal command).

SOLUTION (SQL)

```
mysqldump -u root -p ShopDB > ShopDB_backup.sql
```

NOTE

(Run in Command Prompt / Terminal. Enter your MySQL password when asked.)

Problem 3.3.2 MySQL: restore the backup into a database (terminal command).

SOLUTION (SQL)

```
mysql -u root -p ShopDB < ShopDB_backup.sql
```

NOTE

(Run in Command Prompt / Terminal.)

Problem 3.3.3 SQL Server: full backup and differential backup (reference only).

SOLUTION (SQL)

```
BACKUP DATABASE ShopDB
TO DISK = 'D:\backups\ShopDB.bak';

BACKUP DATABASE ShopDB
TO DISK = 'D:\backups\ShopDB.bak'
WITH DIFFERENTIAL;
```

NOTE

(SQL Server only. A differential backup stores only the changes since the last full backup.)

3.4 CREATE TABLE

Creates a new table. Each column has a name and a data type (see the Data Types section).

SYNTAX

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    ...
);
```

Problem 3.4.1 Create a table 'Persons' with ID, last name, first name, address and city. Then look at its structure.

SOLUTION (SQL)

```
CREATE TABLE Persons (  
    PersonID INT,  
    LastName VARCHAR(255),  
    FirstName VARCHAR(255),  
    Address VARCHAR(255),  
    City VARCHAR(255)  
);  
  
DESCRIBE Persons;
```

RESULT

Query OK, 0 rows affected

Field	Type	Null	Key	Default	Extra
PersonID	int(11)	YES		NULL	
LastName	varchar(255)	YES		NULL	
FirstName	varchar(255)	YES		NULL	
Address	varchar(255)	YES		NULL	
City	varchar(255)	YES		NULL	

5 rows in set

Problem 3.4.2 Insert two people into Persons and read them back.

SOLUTION (SQL)

```
INSERT INTO Persons VALUES (1, 'Shah', 'Aarav', '11 Law Garden', 'Ahmedabad'),  
                             (2, 'Rao', 'Meera', '4 Jubilee Hills', 'Hyderabad');  
  
SELECT * FROM Persons;
```

RESULT

Query OK, 2 rows affected

Records: 2 Duplicates: 0 Warnings: 0

PersonID	LastName	FirstName	Address	City
1	Shah	Aarav	11 Law Garden	Ahmedabad
2	Rao	Meera	4 Jubilee Hills	Hyderabad

2 rows in set

Problem 3.4.3 Create a new table from an existing table (structure + data).

SOLUTION (SQL)

```
CREATE TABLE UKCustomers AS  
SELECT CustomerName, ContactName, City  
FROM Customers  
WHERE Country = 'UK';  
  
SELECT * FROM UKCustomers;
```

RESULT

Query OK, 2 rows affected

Records: 2 Duplicates: 0 Warnings: 0

CustomerName	ContactName	City
London Spice House	James Brown	London
Curry Corner	Sophie Clark	Manchester

2 rows in set

3.5 DROP TABLE and TRUNCATE TABLE

DROP TABLE deletes the table completely (structure + data). TRUNCATE TABLE deletes all rows but keeps the empty table.

SYNTAX

```
DROP TABLE table_name;
TRUNCATE TABLE table_name;
```

Problem 3.5.1 Empty the UKCustomers table but keep its structure.

SOLUTION (SQL)

```
TRUNCATE TABLE UKCustomers;

SELECT COUNT(*) AS RowsLeft FROM UKCustomers;
```

RESULT

```
Query OK, 0 rows affected

+-----+
| RowsLeft |
+-----+
|         0 |
+-----+
1 row in set
```

Problem 3.5.2 Now delete the UKCustomers table completely.

SOLUTION (SQL)

```
DROP TABLE UKCustomers;

SHOW TABLES LIKE 'UKCustomers';
```

RESULT

```
Query OK, 0 rows affected

Empty set
```

Problem 3.5.3 Drop a table only if it exists (safe in scripts).

SOLUTION (SQL)

```
DROP TABLE IF EXISTS ProductsTemplate;
```

RESULT

```
Query OK, 0 rows affected
```

3.6 ALTER TABLE

ALTER TABLE adds, deletes, renames or modifies columns and constraints of an existing table.

SYNTAX

```
ALTER TABLE table_name ADD column_name datatype;
ALTER TABLE table_name DROP COLUMN column_name;
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
ALTER TABLE table_name MODIFY COLUMN column_name datatype; -- MySQL
ALTER TABLE table_name ALTER COLUMN column_name datatype; -- SQL Server
```

Problem 3.6.1 Add an Email column to Persons.

SOLUTION (SQL)

```
ALTER TABLE Persons
ADD Email VARCHAR(255);

DESCRIBE Persons;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

Field	Type	Null	Key	Default	Extra
PersonID	int(11)	YES		NULL	
LastName	varchar(255)	YES		NULL	
FirstName	varchar(255)	YES		NULL	
Address	varchar(255)	YES		NULL	
City	varchar(255)	YES		NULL	
Email	varchar(255)	YES		NULL	

6 rows in set

Problem 3.6.2 Add a DateOfBirth column, then change its type from DATE to YEAR.

SOLUTION (SQL)

```
ALTER TABLE Persons ADD DateOfBirth DATE;

ALTER TABLE Persons
MODIFY COLUMN DateOfBirth YEAR;

DESCRIBE Persons;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

Query OK, 2 rows affected
Records: 2 Duplicates: 0 Warnings: 0

Field	Type	Null	Key	Default	Extra
PersonID	int(11)	YES		NULL	
LastName	varchar(255)	YES		NULL	
FirstName	varchar(255)	YES		NULL	
Address	varchar(255)	YES		NULL	
City	varchar(255)	YES		NULL	
Email	varchar(255)	YES		NULL	
DateOfBirth	year(4)	YES		NULL	

7 rows in set

Problem 3.6.3 Rename the column DateOfBirth to BirthYear.

SOLUTION (SQL)

```
ALTER TABLE Persons
RENAME COLUMN DateOfBirth TO BirthYear;

SELECT * FROM Persons;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

PersonID	LastName	FirstName	Address	City	Email	BirthYear
1	Shah	Aarav	11 Law Garden	Ahmedabad	NULL	NULL
2	Rao	Meera	4 Jubilee Hills	Hyderabad	NULL	NULL

2 rows in set

Problem 3.6.4 Delete the BirthYear column.

SOLUTION (SQL)

```
ALTER TABLE Persons
DROP COLUMN BirthYear;

DESCRIBE Persons;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

Field	Type	Null	Key	Default	Extra
PersonID	int(11)	YES		NULL	
LastName	varchar(255)	YES		NULL	
FirstName	varchar(255)	YES		NULL	
Address	varchar(255)	YES		NULL	
City	varchar(255)	YES		NULL	
Email	varchar(255)	YES		NULL	

6 rows in set

Problem 3.6.5 Rename the whole table from Persons to People and back again.

SOLUTION (SQL)

```
ALTER TABLE Persons RENAME TO People;
SHOW TABLES LIKE 'People';

ALTER TABLE People RENAME TO Persons;
```

RESULT

Query OK, 0 rows affected

Tables_in_ShopDB (People)
People

1 row in set

Query OK, 0 rows affected

3.7 Constraints - overview

Constraints are rules on table data that keep it accurate. NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT and CREATE INDEX. They can be set when a table is created or later with ALTER TABLE.

SYNTAX

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    ...  
);
```

Problem 3.7.1 Create a 'Members' table that uses every kind of constraint at once.

SOLUTION (SQL)

```
CREATE TABLE Members (  
    MemberID INT NOT NULL AUTO_INCREMENT,  
    FullName VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Age INT CHECK (Age >= 18),  
    City VARCHAR(50) DEFAULT 'Ahmedabad',  
    JoinedOn DATE DEFAULT (CURRENT_DATE),  
    ReferredBy INT,  
    PRIMARY KEY (MemberID),  
    FOREIGN KEY (ReferredBy) REFERENCES Members(MemberID)  
);  
  
DESCRIBE Members;
```

RESULT

Query OK, 0 rows affected

Field	Type	Null	Key	Default	Extra
MemberID	int(11)	NO	PRI	NULL	auto_increment
FullName	varchar(100)	NO		NULL	
Email	varchar(100)	YES	UNI	NULL	
Age	int(11)	YES		NULL	
City	varchar(50)	YES		Ahmedabad	
JoinedOn	date	YES		curdate()	
ReferredBy	int(11)	YES	MUL	NULL	

7 rows in set

3.8 NOT NULL

NOT NULL forces a column to always have a value.

SYNTAX

```
column_name datatype NOT NULL  
ALTER TABLE table_name MODIFY column_name datatype NOT NULL; -- MySQL
```

Problem 3.8.1 Insert a valid member (FullName given).

SOLUTION (SQL)

```
INSERT INTO Members (FullName, Email, Age)
VALUES ('Aarav Shah', 'aarav@example.com', 25);

SELECT MemberID, FullName, Email, Age FROM Members;
```

RESULT

Query OK, 1 row affected

MemberID	FullName	Email	Age
1	Aarav Shah	aarav@example.com	25

1 row in set

Problem 3.8.2 Try to insert a member without a name - the database rejects it.

SOLUTION (SQL)

```
INSERT INTO Members (FullName, Email, Age)
VALUES (NULL, 'noname@example.com', 30);
```

RESULT

ERROR 1048 (23000): Column 'FullName' cannot be null

Problem 3.8.3 Make the Address column of Persons mandatory after the table exists.

SOLUTION (SQL)

```
UPDATE Persons SET Address = 'Unknown' WHERE Address IS NULL;
ALTER TABLE Persons MODIFY Address VARCHAR(255) NOT NULL;

DESCRIBE Persons;
```

RESULT

Query OK, 0 rows affected

Rows matched: 0 Changed: 0 Warnings: 0

Query OK, 0 rows affected

Records: 0 Duplicates: 0 Warnings: 0

Field	Type	Null	Key	Default	Extra
PersonID	int(11)	YES		NULL	
LastName	varchar(255)	YES		NULL	
FirstName	varchar(255)	YES		NULL	
Address	varchar(255)	NO		NULL	
City	varchar(255)	YES		NULL	
Email	varchar(255)	YES		NULL	

6 rows in set

3.9 UNIQUE

UNIQUE ensures all values in a column are different. A table can have many UNIQUE constraints but only one PRIMARY KEY.

SYNTAX

```
column_name datatype UNIQUE
ALTER TABLE table_name ADD CONSTRAINT UC_name UNIQUE (col1, col2);
ALTER TABLE table_name DROP INDEX UC_name; -- MySQL
```

Problem 3.9.1 Try to insert a second member with the same email - rejected.

SOLUTION (SQL)

```
INSERT INTO Members (FullName, Email, Age)
VALUES ('Another Aarav', 'aarav@example.com', 40);
```

RESULT

```
ERROR 1062 (23000): Duplicate entry 'aarav@example.com' for key 'Email'
```

Problem 3.9.2 Add a named UNIQUE constraint on Persons (PersonID + LastName combination).

SOLUTION (SQL)

```
ALTER TABLE Persons
ADD CONSTRAINT UC_Person UNIQUE (PersonID, LastName);

SELECT INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'ShopDB' AND TABLE_NAME = 'Persons';
```

RESULT

```
Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

+-----+-----+-----+
| INDEX_NAME | COLUMN_NAME | NON_UNIQUE |
+-----+-----+-----+
| UC_Person | PersonID    | 0          |
| UC_Person | LastName    | 0          |
+-----+-----+-----+
2 rows in set
```

Problem 3.9.3 Remove that UNIQUE constraint again.

SOLUTION (SQL)

```
ALTER TABLE Persons
DROP INDEX UC_Person;

SELECT INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'ShopDB' AND TABLE_NAME = 'Persons';
```

RESULT

```
Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

Empty set
```

3.10 PRIMARY KEY

A PRIMARY KEY uniquely identifies each row. It must be UNIQUE and NOT NULL, and a table can have only one (which may span several columns).

SYNTAX

```
PRIMARY KEY (column)
CONSTRAINT PK_name PRIMARY KEY (col1, col2)
ALTER TABLE table_name ADD PRIMARY KEY (column);
ALTER TABLE table_name DROP PRIMARY KEY;    -- MySQL
```

Problem 3.10.1 Add a primary key to Persons.PersonID after the table was created.

SOLUTION (SQL)

```
ALTER TABLE Persons
ADD PRIMARY KEY (PersonID);

DESCRIBE Persons;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

Field	Type	Null	Key	Default	Extra
PersonID	int(11)	NO	PRI	NULL	
LastName	varchar(255)	YES		NULL	
FirstName	varchar(255)	YES		NULL	
Address	varchar(255)	NO		NULL	
City	varchar(255)	YES		NULL	
Email	varchar(255)	YES		NULL	

6 rows in set

Problem 3.10.2 Try to insert a duplicate PersonID - rejected by the primary key.

SOLUTION (SQL)

```
INSERT INTO Persons (PersonID, LastName, FirstName, Address, City)
VALUES (1, 'Kapoor', 'Riya', '8 Ring Road', 'Surat');
```

RESULT

ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'

Problem 3.10.3 Create a table with a composite (two-column) primary key: one enrolment per student per course.

SOLUTION (SQL)

```
CREATE TABLE Enrolments (
    StudentID INT,
    CourseID INT,
    Grade CHAR(2),
    CONSTRAINT PK_Enrolment PRIMARY KEY (StudentID, CourseID)
);

INSERT INTO Enrolments VALUES (1, 101, 'A'), (1, 102, 'B'), (2, 101, 'A');
SELECT * FROM Enrolments;
```

RESULT

Query OK, 0 rows affected, 1 warning
Query OK, 3 rows affected
Records: 3 Duplicates: 0 Warnings: 0

StudentID	CourseID	Grade
1	101	A
1	102	B
2	101	A

3 rows in set

Problem 3.10.4 Drop the primary key of Enrolments.

SOLUTION (SQL)

```
ALTER TABLE Enrolments DROP PRIMARY KEY;

SELECT INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'ShopDB' AND TABLE_NAME = 'Enrolments';
```

RESULT

```
Query OK, 3 rows affected
Records: 3 Duplicates: 0 Warnings: 0

Empty set
```

3.11 FOREIGN KEY

A FOREIGN KEY links a column to the PRIMARY KEY of another table and prevents actions that would break that link (e.g. orders for customers that do not exist).

SYNTAX

```
FOREIGN KEY (column) REFERENCES parent_table(parent_column)
ALTER TABLE child ADD CONSTRAINT FK_name FOREIGN KEY (col) REFERENCES parent(col);
ALTER TABLE child DROP FOREIGN KEY FK_name; -- MySQL
```

Problem 3.11.1 See the foreign keys already defined on the Orders table.

SOLUTION (SQL)

```
SELECT CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'ShopDB' AND TABLE_NAME = 'Orders'
AND REFERENCED_TABLE_NAME IS NOT NULL;
```

RESULT

```
+-----+-----+-----+-----+
| CONSTRAINT_NAME | COLUMN_NAME | REFERENCED_TABLE_NAME | REFERENCED_COLUMN_NAME |
+-----+-----+-----+-----+
| Orders_ibfk_1   | CustomerID  | Customers              | CustomerID              |
| Orders_ibfk_2   | EmployeeID  | Employees              | EmployeeID              |
| Orders_ibfk_3   | ShipperID   | Shippers               | ShipperID               |
+-----+-----+-----+-----+
3 rows in set
```

Problem 3.11.2 Try to create an order for customer 999 who does not exist - the foreign key blocks it.

SOLUTION (SQL)

```
INSERT INTO Orders (CustomerID, EmployeeID, OrderDate, ShipperID)
VALUES (999, 1, '2026-07-01', 1);
```

RESULT

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`ShopDB`.`Orders`, CONSTRAINT `Orders_ibfk_1` FOREIGN KEY (`CustomerID`) REFERENCES `Customers`
(`CustomerID`))
```

Problem 3.11.3 Add a named foreign key: Enrolments.StudentID must exist in a new Students table.

SOLUTION (SQL)

```
CREATE TABLE Students (StudentID INT PRIMARY KEY, Name VARCHAR(50));
INSERT INTO Students VALUES (1, 'Aarav'), (2, 'Meera');

ALTER TABLE Enrolments
ADD CONSTRAINT FK_StudentEnrol
FOREIGN KEY (StudentID) REFERENCES Students(StudentID);

SELECT e.StudentID, s.Name, e.CourseID, e.Grade
FROM Enrolments e JOIN Students s ON e.StudentID = s.StudentID;
```

RESULT

```
Query OK, 0 rows affected

Query OK, 2 rows affected
Records: 2 Duplicates: 0 Warnings: 0

Query OK, 3 rows affected
Records: 3 Duplicates: 0 Warnings: 0

+-----+-----+-----+-----+
| StudentID | Name | CourseID | Grade |
+-----+-----+-----+-----+
| 1 | Aarav | 101 | A |
| 1 | Aarav | 102 | B |
| 2 | Meera | 101 | A |
+-----+-----+-----+-----+
3 rows in set
```

Problem 3.11.4 Drop that foreign key.

SOLUTION (SQL)

```
ALTER TABLE Enrolments
DROP FOREIGN KEY FK_StudentEnrol;
```

RESULT

```
Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0
```

3.12 CHECK

CHECK limits the values allowed in a column (supported and enforced in MySQL 8.0.16+ and MariaDB 10.2+).

SYNTAX

```
column datatype CHECK (condition)
ALTER TABLE table_name ADD CONSTRAINT CHK_name CHECK (condition);
ALTER TABLE table_name DROP CHECK CHK_name;    -- MySQL
```

Problem 3.12.1 Try to add a 16-year-old member - the CHECK (Age >= 18) rule rejects it.

SOLUTION (SQL)

```
INSERT INTO Members (FullName, Email, Age)
VALUES ('Young Kid', 'kid@example.com', 16);
```

RESULT

```
ERROR 4025 (23000): CONSTRAINT `Members.Age` failed for `ShopDB`.`Members`
```

Problem 3.12.2 Add a named CHECK constraint: every product price must be above zero.

SOLUTION (SQL)

```
ALTER TABLE Products
ADD CONSTRAINT CHK_Price CHECK (Price > 0);

UPDATE Products SET Price = -5 WHERE ProductID = 1;
```

RESULT

```
Query OK, 13 rows affected
Records: 13 Duplicates: 0 Warnings: 0

ERROR 4025 (23000): CONSTRAINT `CHK_Price` failed for `ShopDB`.`Products`
```

Problem 3.12.3 Drop the CHK_Price constraint.

SOLUTION (SQL)

```
ALTER TABLE Products
DROP CONSTRAINT CHK_Price;
```

RESULT

```
Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0
```

3.13 DEFAULT

DEFAULT sets a value that is used automatically when no value is given on INSERT.

SYNTAX

```
column datatype DEFAULT value
ALTER TABLE table_name ALTER column_name SET DEFAULT value;    -- MySQL
ALTER TABLE table_name ALTER column_name DROP DEFAULT;         -- MySQL
```

Problem 3.13.1 Insert a member without City and JoinedOn - the defaults are applied.

SOLUTION (SQL)

```
INSERT INTO Members (FullName, Email, Age)
VALUES ('Meera Rao', 'meera@example.com', 29);

SELECT FullName, City, JoinedOn FROM Members WHERE FullName = 'Meera Rao';
```

RESULT

```
Query OK, 1 row affected

+-----+-----+-----+
| FullName | City      | JoinedOn |
+-----+-----+-----+
| Meera Rao | Ahmedabad | 2026-09-25 |
+-----+-----+-----+
1 row in set
```

Tip: JoinedOn shows the date the query was executed (CURRENT_DATE).

Problem 3.13.2 Give Persons.City a default value of 'Ahmedabad', then insert without a city.

SOLUTION (SQL)

```
ALTER TABLE Persons
ALTER City SET DEFAULT 'Ahmedabad';

INSERT INTO Persons (PersonID, LastName, FirstName, Address)
VALUES (3, 'Patel', 'Dev', '22 SG Highway');

SELECT * FROM Persons;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

Query OK, 1 row affected

PersonID	LastName	FirstName	Address	City	Email
1	Shah	Aarav	11 Law Garden	Ahmedabad	NULL
2	Rao	Meera	4 Jubilee Hills	Hyderabad	NULL
3	Patel	Dev	22 SG Highway	Ahmedabad	NULL

3 rows in set

Problem 3.13.3 Remove the default from Persons.City.

SOLUTION (SQL)

```
ALTER TABLE Persons
ALTER City DROP DEFAULT;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

3.14 CREATE INDEX

Indexes make searching faster (like the index at the back of a book) but make INSERT/UPDATE slightly slower. Index columns you search on often. To list indexes you can also run SHOW INDEX FROM table_name; (same information, many more columns).

SYNTAX

```
CREATE INDEX index_name ON table_name (column1, column2);
CREATE UNIQUE INDEX index_name ON table_name (column);
ALTER TABLE table_name DROP INDEX index_name; -- MySQL
DROP INDEX table_name.index_name; -- SQL Server
```

Problem 3.14.1 Create an index on Customers.City because we often filter by city.

SOLUTION (SQL)

```
CREATE INDEX idx_customer_city
ON Customers (City);

SELECT INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'ShopDB' AND TABLE_NAME = 'Customers'
AND INDEX_NAME = 'idx_customer_city';
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

INDEX_NAME	COLUMN_NAME	NON_UNIQUE
idx_customer_city	City	1

1 row in set

Problem 3.14.2 Create a composite index on Employees (LastName, FirstName).

SOLUTION (SQL)

```
CREATE INDEX idx_emp_name
ON Employees (LastName, FirstName);

SELECT INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'ShopDB' AND TABLE_NAME = 'Employees'
AND INDEX_NAME = 'idx_emp_name';
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

INDEX_NAME	COLUMN_NAME	NON_UNIQUE
idx_emp_name	LastName	1
idx_emp_name	FirstName	1

2 rows in set

Problem 3.14.3 Create a UNIQUE index so no two shippers can have the same phone number.

SOLUTION (SQL)

```
CREATE UNIQUE INDEX idx_shipper_phone
ON Shippers (Phone);

SELECT INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'ShopDB' AND TABLE_NAME = 'Shippers'
AND INDEX_NAME = 'idx_shipper_phone';
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

INDEX_NAME	COLUMN_NAME	NON_UNIQUE
idx_shipper_phone	Phone	0

1 row in set

Problem 3.14.4 Use EXPLAIN to see which index a query uses.

SOLUTION (SQL)

```
EXPLAIN SELECT * FROM Customers WHERE City = 'Ahmedabad';
```

RESULT

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	Customers	ref	idx_customer_city	idx_customer_city	203	const	2	Using index condition

1 row in set

Problem 3.14.5 Drop the index on Employees.

SOLUTION (SQL)

```
ALTER TABLE Employees  
DROP INDEX idx_emp_name;
```

RESULT

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

3.15 AUTO_INCREMENT

AUTO_INCREMENT generates a unique number automatically for each new row (usually the primary key). SQL Server uses IDENTITY(1,1), MS Access uses AUTOINCREMENT, Oracle uses sequences or IDENTITY.

SYNTAX

column INT NOT NULL AUTO_INCREMENT PRIMARY KEY	-- MySQL
ALTER TABLE table_name AUTO_INCREMENT = 100;	-- start value
column INT IDENTITY(1,1) PRIMARY KEY	-- SQL Server

Problem 3.15.1 Create a Tickets table whose IDs start at 100, insert two rows without giving an ID.

SOLUTION (SQL)

```
CREATE TABLE Tickets (  
    TicketID INT NOT NULL AUTO_INCREMENT,  
    Subject VARCHAR(100),  
    PRIMARY KEY (TicketID)  
);  
ALTER TABLE Tickets AUTO_INCREMENT = 100;  
  
INSERT INTO Tickets (Subject) VALUES ('Late delivery'), ('Wrong item received');  
  
SELECT * FROM Tickets;
```

RESULT

Query OK, 0 rows affected

Query OK, 0 rows affected
Records: 0 Duplicates: 0 Warnings: 0

Query OK, 2 rows affected
Records: 2 Duplicates: 0 Warnings: 0

TicketID	Subject
100	Late delivery
101	Wrong item received

2 rows in set

Problem 3.15.2 Find out which ID was generated by the last insert.

SOLUTION (SQL)

```
INSERT INTO Tickets (Subject) VALUES ('Refund request');
SELECT LAST_INSERT_ID() AS NewTicketID;
```

RESULT

Query OK, 1 row affected

```
+-----+
| NewTicketID |
+-----+
|          102 |
+-----+
1 row in set
```

3.16 Dates

Dates must match the column format. MySQL types: DATE (YYYY-MM-DD), DATETIME / TIMESTAMP (YYYY-MM-DD HH:MI:SS), YEAR. Keep time out of DATE comparisons, or use DATE() to strip it.

Problem 3.16.1 Find orders placed on 14 February 2026.

SOLUTION (SQL)

```
SELECT * FROM Orders
WHERE OrderDate = '2026-02-14';
```

RESULT

```
+-----+-----+-----+-----+-----+
| OrderID | CustomerID | EmployeeID | OrderDate | ShipperID |
+-----+-----+-----+-----+-----+
|    1005 |          5 |          5 | 2026-02-14 |          2 |
+-----+-----+-----+-----+-----+
1 row in set
```

Problem 3.16.2 Find all orders placed in March 2026 using YEAR() and MONTH().

SOLUTION (SQL)

```
SELECT OrderID, OrderDate
FROM Orders
WHERE YEAR(OrderDate) = 2026 AND MONTH(OrderDate) = 3;
```

RESULT

```
+-----+-----+
| OrderID | OrderDate |
+-----+-----+
|    1007 | 2026-03-04 |
|    1008 | 2026-03-15 |
|    1009 | 2026-03-28 |
+-----+-----+
3 rows in set
```

Problem 3.16.3

 How many orders were placed in each month?

SOLUTION (SQL)

```
SELECT DATE_FORMAT(OrderDate, '%Y-%m') AS Month, COUNT(*) AS Orders
FROM Orders
GROUP BY DATE_FORMAT(OrderDate, '%Y-%m')
ORDER BY Month;
```

RESULT

```
+-----+-----+
| Month | Orders |
+-----+-----+
| 2026-01 | 3 |
| 2026-02 | 3 |
| 2026-03 | 3 |
| 2026-04 | 2 |
| 2026-05 | 2 |
| 2026-06 | 2 |
+-----+-----+
6 rows in set
```

Problem 3.16.4

 How old was each employee on 1 January 2026?

SOLUTION (SQL)

```
SELECT FirstName, BirthDate,
TIMESTAMPDIFF(YEAR, BirthDate, '2026-01-01') AS AgeOn2026
FROM Employees;
```

RESULT

```
+-----+-----+-----+
| FirstName | BirthDate | AgeOn2026 |
+-----+-----+-----+
| Arjun | 1980-05-14 | 45 |
| Kavya | 1988-11-02 | 37 |
| Rohan | 1992-03-21 | 33 |
| Sara | 1995-07-30 | 30 |
| Vikram | 1990-01-09 | 35 |
| Neha | 1998-12-15 | 27 |
+-----+-----+-----+
6 rows in set
```

Problem 3.16.5

 Show how many days passed between each order and 30 June 2026.

SOLUTION (SQL)

```
SELECT OrderID, OrderDate,
DATEDIFF('2026-06-30', OrderDate) AS DaysAgo
FROM Orders
WHERE OrderID > 1011;
```

RESULT

```
+-----+-----+-----+
| OrderID | OrderDate | DaysAgo |
+-----+-----+-----+
| 1012 | 2026-05-02 | 59 |
| 1013 | 2026-05-20 | 41 |
| 1014 | 2026-06-01 | 29 |
| 1015 | 2026-06-11 | 19 |
+-----+-----+-----+
4 rows in set
```

3.17 VIEWS

A view is a virtual table based on a SELECT. It always shows up-to-date data and can hide complex joins.

SYNTAX

```
CREATE VIEW view_name AS SELECT ... ;
CREATE OR REPLACE VIEW view_name AS SELECT ... ;
DROP VIEW view_name;
```

Problem 3.17.1 Create a view listing Indian customers and query it like a table.

SOLUTION (SQL)

```
CREATE VIEW vw_IndianCustomers AS
SELECT CustomerName, ContactName, City
FROM Customers
WHERE Country = 'India';

SELECT * FROM vw_IndianCustomers;
```

RESULT

Query OK, 0 rows affected

CustomerName	ContactName	City
Sharma General Store	Rakesh Sharma	Ahmedabad
Patel Supermart	Nisha Patel	Ahmedabad
Mehta Kirana	Anil Mehta	Surat
Fresh Basket	Priya Nair	Bengaluru
Desi Mart	Imran Khan	Mumbai
Kolkata Sweets	Sourav Das	Kolkata
Pune Pantry	Rahul Joshi	Pune

7 rows in set

Problem 3.17.2 Create a view of products priced above the average price.

SOLUTION (SQL)

```
CREATE VIEW vw_AboveAvgProducts AS
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);

SELECT * FROM vw_AboveAvgProducts ORDER BY Price DESC;
```

RESULT

Query OK, 0 rows affected

ProductName	Price
Basmati Rice	780.00
Ghee	650.00
Kaju Katli	540.00
Whole Wheat Atta	460.00
Saffron	438.90
Filter Coffee	320.00

6 rows in set

Problem 3.17.3 Create a reporting view that hides a 4-table join: order totals per order.

SOLUTION (SQL)

```
CREATE VIEW vw_OrderTotals AS
SELECT o.OrderID, o.OrderDate, c.CustomerName, c.Country,
       SUM(od.Quantity * p.Price) AS OrderTotal
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY o.OrderID, o.OrderDate, c.CustomerName, c.Country;

SELECT * FROM vw_OrderTotals ORDER BY OrderTotal DESC LIMIT 5;
```

RESULT

Query OK, 0 rows affected

OrderID	OrderDate	CustomerName	Country	OrderTotal
1008	2026-03-15	Dubai Gourmet	UAE	38400.00
1014	2026-06-01	Sharjah Traders	UAE	28260.00
1010	2026-04-06	New York Masala	USA	19111.20
1003	2026-01-20	London Spice House	UK	15314.00
1005	2026-02-14	Desi Mart	India	9720.00

5 rows in set

Problem 3.17.4 Update the Indian customers view to also show PostalCode (CREATE OR REPLACE).

SOLUTION (SQL)

```
CREATE OR REPLACE VIEW vw_IndianCustomers AS
SELECT CustomerName, City, PostalCode
FROM Customers
WHERE Country = 'India';

SELECT * FROM vw_IndianCustomers LIMIT 3;
```

RESULT

Query OK, 0 rows affected

CustomerName	City	PostalCode
Sharma General Store	Ahmedabad	380001
Patel Supermart	Ahmedabad	380009
Mehta Kirana	Surat	395003

3 rows in set

Problem 3.17.5 Drop the above-average view.

SOLUTION (SQL)

```
DROP VIEW vw_AboveAvgProducts;
```

RESULT

Query OK, 0 rows affected

3.18 SQL Injection

SQL injection happens when user input is glued directly into a SQL string. An attacker types SQL instead of data and changes the meaning of the query. Below we simulate what the application would build from bad input.

SYNTAX

```
-- DANGEROUS application code (pseudo-code)
txtUserId = getRequestString("UserId");
sql = "SELECT * FROM Customers WHERE CustomerID = " + txtUserId;
```

Problem 3.18.1 Normal input '5' builds a harmless query.

SOLUTION (SQL)

```
SELECT CustomerID, CustomerName FROM Customers WHERE CustomerID = 5;
```

RESULT

```
+-----+-----+
| CustomerID | CustomerName |
+-----+-----+
|          5 | Desi Mart    |
+-----+-----+
1 row in set
```

Problem 3.18.2 Attacker types 105 OR 1=1 -> the condition is ALWAYS true, so every customer leaks.

SOLUTION (SQL)

```
SELECT CustomerID, CustomerName FROM Customers WHERE CustomerID = 105 OR 1=1;
```

RESULT

```
+-----+-----+
| CustomerID | CustomerName |
+-----+-----+
|          1 | Sharma General Store |
|          2 | Patel Supermart    |
|          3 | Mehta Kirana       |
|          4 | Fresh Basket       |
|          5 | Desi Mart          |
|          6 | London Spice House |
|          7 | Curry Corner       |
|          8 | Berlin Bazaar      |
|          9 | Munich Deli        |
|         10 | Dubai Gourmet      |
|         11 | Sharjah Traders    |
|         12 | New York Masala    |
|         13 | Texas Tandoor      |
|         14 | Kolkata Sweets     |
|         16 | Pune Pantry        |
+-----+-----+
15 rows in set
```

Problem 3.18.3 Login bypass: attacker enters "" or ""="" as both username and password.

SOLUTION (SQL)

```
SELECT * FROM Users WHERE Name = "" or ""="" AND Pass = "" or ""="";
```

NOTE

(""="" is always true, so the query returns every user and the attacker logs in.)

Problem 3.18.4 Batched statements: input 105; DROP TABLE Suppliers would run TWO statements.

SOLUTION (SQL)

```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

NOTE

(Not executed! The fix for all of these is parameterised queries - next section.)

3.19 SQL Parameters and Prepared Statements

The fix for SQL injection: send the SQL and the values separately. The database treats values purely as data, never as SQL code. MySQL supports this directly with PREPARE / EXECUTE; every programming language has an equivalent.

SYNTAX

```
PREPARE stmt_name FROM 'SELECT ... WHERE col = ?';
SET @var = value;
EXECUTE stmt_name USING @var;
DEALLOCATE PREPARE stmt_name;
```

Problem 3.19.1 Prepare a customer lookup with a ? placeholder and run it for customer 5.

SOLUTION (SQL)

```
PREPARE findCustomer FROM
  'SELECT CustomerID, CustomerName, City FROM Customers WHERE CustomerID = ?';

SET @id = 5;
EXECUTE findCustomer USING @id;

DEALLOCATE PREPARE findCustomer;
```

RESULT

```
Query OK, 0 rows affected
Statement prepared

Query OK, 0 rows affected

+-----+-----+-----+
| CustomerID | CustomerName | City |
+-----+-----+-----+
|          5 | Desi Mart    | Mumbai |
+-----+-----+-----+
1 row in set

Query OK, 0 rows affected
```

Problem 3.19.2 Pass the attack string '105 OR 1=1' as a parameter - it is treated as a plain value, so nothing leaks.

SOLUTION (SQL)

```
PREPARE findCustomer FROM
  'SELECT CustomerID, CustomerName FROM Customers WHERE CustomerName = ?';

SET @input = '105 OR 1=1';
EXECUTE findCustomer USING @input;

DEALLOCATE PREPARE findCustomer;
```

RESULT

```
Query OK, 0 rows affected
Statement prepared

Query OK, 0 rows affected

Empty set

Query OK, 0 rows affected
```

Problem 3.19.3 Prepared statement with two parameters: products in a category under a price.

SOLUTION (SQL)

```
PREPARE cheapInCat FROM
  'SELECT ProductName, Price FROM Products WHERE CategoryID = ? AND Price < ?';

SET @cat = 1, @maxPrice = 300;
EXECUTE cheapInCat USING @cat, @maxPrice;

DEALLOCATE PREPARE cheapInCat;
```

RESULT

```
Query OK, 0 rows affected
Statement prepared

Query OK, 0 rows affected

+-----+-----+
| ProductName | Price |
+-----+-----+
| Masala Chai | 250.00 |
| Mango Juice | 120.00 |
+-----+-----+
2 rows in set

Query OK, 0 rows affected
```

Problem 3.19.4 The same safe query from a Java (JDBC) application.

SOLUTION (SQL)

```
String sql = "SELECT CustomerName, City FROM Customers WHERE CustomerID = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, Integer.parseInt(userInput));
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        System.out.println(rs.getString("CustomerName"));
    }
}
```

NOTE

(Java code - never build SQL with string concatenation like "... = " + userInput.)

3.20 SQL Data Types (MySQL reference)

Choose the smallest type that safely fits the data. Types used in ShopDB are marked in bold.

Type	Category	Stores	Example
INT	Numeric	Whole numbers, about +/- 2.1 billion	CustomerID INT
TINYINT / SMALLINT / BIGINT	Numeric	Smaller / larger whole numbers	Age TINYINT
DECIMAL(p,s)	Numeric	Exact decimals - use for money	Price DECIMAL(10,2)
FLOAT / DOUBLE	Numeric	Approximate decimals (scientific values)	Rating FLOAT
BOOLEAN	Numeric	Alias for TINYINT(1): 0 = false, 1 = true	IsActive BOOLEAN
CHAR(n)	String	Fixed length text, padded	Grade CHAR(2)
VARCHAR(n)	String	Variable length text up to n characters	City VARCHAR(50)
TEXT / LONGTEXT	String	Long text (descriptions, articles)	Notes TEXT
ENUM('a','b')	String	One value from a fixed list	Size ENUM('S','M','L')
DATE	Date/Time	YYYY-MM-DD	OrderDate DATE
DATETIME / TIMESTAMP	Date/Time	YYYY-MM-DD HH:MI:SS	CreatedAt DATETIME
TIME	Date/Time	HH:MI:SS	OpensAt TIME
YEAR	Date/Time	4-digit year	BirthYear YEAR

Type	Category	Stores	Example
BLOB / JSON	Other	Binary data / JSON documents	Settings JSON

SQL Server equivalents: NVARCHAR for Unicode text, BIT for booleans, DATETIME2 for date+time, MONEY for currency. Oracle: NUMBER, VARCHAR2, DATE.

SQL Functions - Hands-On

MySQL string, numeric, date and advanced functions from the W3Schools reference

4.1 String Functions

MySQL string functions for cleaning and formatting text.

Problem 4.1.1 CONCAT / CONCAT_WS: build a display label and a comma-separated line.

SOLUTION (SQL)

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName,
       CONCAT_WS(' | ', FirstName, LastName, Notes) AS Profile
FROM Employees
LIMIT 3;
```

RESULT

```
+-----+-----+
| FullName      | Profile                                     |
+-----+-----+
| Arjun Mehta   | Arjun | Mehta | Sales Head |
| Kavya Iyer    | Kavya | Iyer | Senior sales, South region |
| Rohan Desai   | Rohan | Desai | Sales executive, West region |
+-----+-----+
3 rows in set
```

Problem 4.1.2 UPPER / LOWER (also UCASE / LCASE): normalise text case.

SOLUTION (SQL)

```
SELECT UPPER(CustomerName) AS Upper_Name, LOWER(City) AS lower_city,
       UCASE(Country) AS UCase_Country
FROM Customers
LIMIT 3;
```

RESULT

```
+-----+-----+-----+
| Upper_Name      | lower_city | UCase_Country |
+-----+-----+-----+
| SHARMA GENERAL STORE | ahmedabad | INDIA |
| PATEL SUPERMART    | ahmedabad | INDIA |
| MEHTA KIRANA       | surat     | INDIA |
+-----+-----+-----+
3 rows in set
```

Problem 4.1.3 LENGTH / CHAR_LENGTH: how long is each product name?

SOLUTION (SQL)

```
SELECT ProductName, LENGTH(ProductName) AS Bytes, CHAR_LENGTH(ProductName) AS Chars
FROM Products
ORDER BY Chars DESC
LIMIT 4;
```

RESULT

```
+-----+-----+-----+
| ProductName      | Bytes | Chars |
+-----+-----+-----+
| Whole Wheat Atta | 16    | 16    |
| Gulab Jamun Mix  | 15    | 15    |
| Bhujia Namkeen   | 14    | 14    |
| Filter Coffee    | 13    | 13    |
+-----+-----+-----+
4 rows in set
```

Problem 4.1.4 SUBSTRING / LEFT / RIGHT / MID: create short codes from names.

SOLUTION (SQL)

```
SELECT CustomerName,
       LEFT(CustomerName, 3)      AS First3,
       RIGHT(CustomerName, 4)     AS Last4,
       SUBSTRING(CustomerName, 1, 5) AS Sub1to5,
       MID(CustomerName, 3, 4)    AS Mid3_4
FROM Customers
LIMIT 4;
```

RESULT

CustomerName	First3	Last4	Sub1to5	Mid3_4
Sharma General Store	Sha	tore	Sharm	arma
Patel Supermart	Pat	mart	Patel	tel
Mehta Kirana	Meh	rana	Mehta	hta
Fresh Basket	Fre	sket	Fresh	esh

4 rows in set

Problem 4.1.5 REPLACE / REVERSE / REPEAT: transform text.

SOLUTION (SQL)

```
SELECT ProductName,
       REPLACE(ProductName, ' ', '_') AS Slug,
       REVERSE(ProductName)          AS Reversed,
       REPEAT('*', 3)                 AS Stars
FROM Products
LIMIT 3;
```

RESULT

ProductName	Slug	Reversed	Stars
Masala Chai	Masala_Chai	iahC alasaM	***
Filter Coffee	Filter_Coffee	eeffoC retliF	***
Mango Juice	Mango_Juice	eciuJ ognam	***

3 rows in set

Problem 4.1.6 TRIM / LTRIM / RTRIM: remove unwanted spaces.

SOLUTION (SQL)

```
SELECT CONCAT('[', TRIM(' Masala '), '])' AS Trimmed,
       CONCAT('[', LTRIM(' Masala '), '])' AS LeftTrimmed,
       CONCAT('[', RTRIM(' Masala '), '])' AS RightTrimmed;
```

RESULT

Trimmed	LeftTrimmed	RightTrimmed
[Masala]	[Masala]	[Masala]

1 row in set

Problem 4.1.7 LPAD / RPAD: format order IDs and pad names to fixed width.

SOLUTION (SQL)

```
SELECT LPAD(OrderID, 8, '0')      AS PaddedID,
       RPAD(ShipperID, 4, '-')    AS RightPad
FROM Orders
LIMIT 3;
```

RESULT

```
+-----+-----+
| PaddedID | RightPad |
+-----+-----+
| 00001002 | 1---    |
| 00001004 | 1---    |
| 00001009 | 1---    |
+-----+-----+
3 rows in set
```

Problem 4.1.8 INSTR / LOCATE / POSITION: where does a word appear inside a name?

SOLUTION (SQL)

```
SELECT CustomerName,
       INSTR(CustomerName, 'a')      AS Instr_a,
       LOCATE('Mart', CustomerName)  AS Locate_Mart,
       POSITION('Store' IN CustomerName) AS Pos_Store
FROM Customers
WHERE CustomerID IN (1, 2, 5);
```

RESULT

```
+-----+-----+-----+-----+
| CustomerName | Instr_a | Locate_Mart | Pos_Store |
+-----+-----+-----+-----+
| Sharma General Store | 3 | 0 | 16 |
| Patel Supermart | 2 | 12 | 0 |
| Desi Mart | 7 | 6 | 0 |
+-----+-----+-----+-----+
3 rows in set
```

Problem 4.1.9 SUBSTRING_INDEX: split a text by a delimiter (first name / last name from ContactName).

SOLUTION (SQL)

```
SELECT ContactName,
       SUBSTRING_INDEX(ContactName, ' ', 1) AS FirstName,
       SUBSTRING_INDEX(ContactName, ' ', -1) AS LastName
FROM Customers
LIMIT 4;
```

RESULT

```
+-----+-----+-----+
| ContactName | FirstName | LastName |
+-----+-----+-----+
| Rakesh Sharma | Rakesh | Sharma |
| Nisha Patel | Nisha | Patel |
| Anil Mehta | Anil | Mehta |
| Priya Nair | Priya | Nair |
+-----+-----+-----+
4 rows in set
```

Problem 4.1.10 FORMAT / FIELD / FIND_IN_SET / STRCMP / ASCII / SPACE.

SOLUTION (SQL)

```
SELECT FORMAT(1234567.891, 2) AS Formatted,
       FIELD('UK', 'India', 'UK', 'USA') AS FieldPos,
       FIND_IN_SET('b', 'a,b,c') AS SetPos,
       STRCMP('Apple', 'Banana') AS Cmp,
       ASCII('A') AS AsciiA,
       CONCAT('A', SPACE(3), 'B') AS Spaced;
```

RESULT

Formatted	FieldPos	SetPos	Cmp	AsciiA	Spaced
1,234,567.89	2	2	-1	65	A B

1 row in set

4.2 Numeric Functions

Math functions for calculations on numeric columns.

Problem 4.2.1 ROUND / CEIL / FLOOR / TRUNCATE: different ways to cut decimals (price with 18% GST).

SOLUTION (SQL)

```
SELECT ProductName, Price * 1.18 AS WithGST,
       ROUND(Price * 1.18, 1) AS Rounded,
       CEIL(Price * 1.18) AS Ceil,
       FLOOR(Price * 1.18) AS Floor,
       TRUNCATE(Price * 1.18, 1) AS Truncated
FROM Products
WHERE CategoryID = 5;
```

RESULT

ProductName	WithGST	Rounded	Ceil	Floor	Truncated
Garam Masala	110.3300	110.3	111	110	110.3
Saffron	517.9020	517.9	518	517	517.9

2 rows in set

Problem 4.2.2 ABS / SIGN / MOD / POWER / SQRT / PI.

SOLUTION (SQL)

```
SELECT ABS(-250) AS AbsVal,
       SIGN(-250) AS SignVal,
       MOD(17, 5) AS ModVal,
       POWER(2, 10) AS Pow,
       SQRT(144) AS SqRoot,
       ROUND(PI(), 4) AS PiVal;
```

RESULT

AbsVal	SignVal	ModVal	Pow	SqRoot	PiVal
250	-1	2	1024	12	3.1416

1 row in set

Problem 4.2.3 GREATEST / LEAST: compare values across columns in one row.

SOLUTION (SQL)

```
SELECT GREATEST(120, 450, 90) AS Biggest,  
       LEAST(120, 450, 90)   AS Smallest;
```

RESULT

```
+-----+-----+  
| Biggest | Smallest |  
+-----+-----+  
|      450 |       90 |  
+-----+-----+  
1 row in set
```

Problem 4.2.4 How far is each product's price from the average price? (ABS + subquery)

SOLUTION (SQL)

```
SELECT ProductName, Price,  
       ROUND(ABS(Price - (SELECT AVG(Price) FROM Products)), 2) AS DistanceFromAvg  
FROM Products  
ORDER BY DistanceFromAvg  
LIMIT 4;
```

RESULT

```
+-----+-----+-----+  
| ProductName | Price | DistanceFromAvg |  
+-----+-----+-----+  
| Filter Coffee | 320.00 | 10.58 |  
| Masala Chai | 250.00 | 59.42 |  
| Saffron | 438.90 | 129.48 |  
| Whole Wheat Atta | 460.00 | 150.58 |  
+-----+-----+-----+  
4 rows in set
```

Problem 4.2.5 RAND: pick 2 random products (the result changes every time you run it).

SOLUTION (SQL)

```
SELECT ProductName FROM Products  
ORDER BY RAND()  
LIMIT 2;
```

RESULT

```
+-----+  
| ProductName |  
+-----+  
| Bhujia Namkeen |  
| Saffron |  
+-----+  
2 rows in set
```

Tip: Your output will differ - that is expected.

4.3 Date Functions

Functions to get, format and calculate dates.

Problem 4.3.1 NOW / CURDATE / CURTIME: current date and time (your values will differ).

SOLUTION (SQL)

```
SELECT NOW() AS NowValue, CURDATE() AS Today, CURTIME() AS TimeNow;
```

RESULT

```
+-----+-----+-----+  
| NowValue | Today | TimeNow |  
+-----+-----+-----+  
| 2026-09-25 12:25:20 | 2026-09-25 | 12:25:20 |  
+-----+-----+-----+  
1 row in set
```

Tip: Output depends on when you run it.

Problem 4.3.2 DATE_FORMAT: show order dates in Indian style dd-mm-yyyy and with day names.

SOLUTION (SQL)

```
SELECT OrderID,
       DATE_FORMAT(OrderDate, '%d-%m-%Y') AS IndianFormat,
       DATE_FORMAT(OrderDate, '%W, %d %M %Y') AS LongFormat
FROM Orders
LIMIT 3;
```

RESULT

```
+-----+-----+-----+
| OrderID | IndianFormat | LongFormat |
+-----+-----+-----+
| 1001 | 05-01-2026 | Monday, 05 January 2026 |
| 1002 | 12-01-2026 | Monday, 12 January 2026 |
| 1003 | 20-01-2026 | Tuesday, 20 January 2026 |
+-----+-----+-----+
3 rows in set
```

Problem 4.3.3 DAY / MONTH / YEAR / DAYNAME / MONTHNAME / QUARTER / WEEK.

SOLUTION (SQL)

```
SELECT OrderDate, DAY(OrderDate) AS D, MONTH(OrderDate) AS M, YEAR(OrderDate) AS Y,
       DAYNAME(OrderDate) AS DayName, MONTHNAME(OrderDate) AS MonthName,
       QUARTER(OrderDate) AS Q, WEEK(OrderDate) AS Wk
FROM Orders
LIMIT 4;
```

RESULT

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| OrderDate | D | M | Y | DayName | MonthName | Q | Wk |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2026-01-05 | 5 | 1 | 2026 | Monday | January | 1 | 1 |
| 2026-01-12 | 12 | 1 | 2026 | Monday | January | 1 | 2 |
| 2026-01-20 | 20 | 1 | 2026 | Tuesday | January | 1 | 3 |
| 2026-02-03 | 3 | 2 | 2026 | Tuesday | February | 1 | 5 |
+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set
```

Problem 4.3.4 DATE_ADD / DATE_SUB / ADDDATE: expected delivery 7 days after order, and 1 month before.

SOLUTION (SQL)

```
SELECT OrderID, OrderDate,
       DATE_ADD(OrderDate, INTERVAL 7 DAY) AS ExpectedDelivery,
       DATE_SUB(OrderDate, INTERVAL 1 MONTH) AS OneMonthBefore,
       ADDDATE(OrderDate, 30) AS PaymentDue
FROM Orders
LIMIT 3;
```

RESULT

```
+-----+-----+-----+-----+-----+
| OrderID | OrderDate | ExpectedDelivery | OneMonthBefore | PaymentDue |
+-----+-----+-----+-----+-----+
| 1001 | 2026-01-05 | 2026-01-12 | 2025-12-05 | 2026-02-04 |
| 1002 | 2026-01-12 | 2026-01-19 | 2025-12-12 | 2026-02-11 |
| 1003 | 2026-01-20 | 2026-01-27 | 2025-12-20 | 2026-02-19 |
+-----+-----+-----+-----+-----+
3 rows in set
```

Problem 4.3.5 LAST_DAY / DAYOFWEEK / DAYOFYEAR / EXTRACT.

SOLUTION (SQL)

```
SELECT OrderDate,
       LAST_DAY(OrderDate)           AS MonthEnd,
       DAYOFWEEK(OrderDate)          AS DayOfWeekNo,
       DAYOFYEAR(OrderDate)          AS DayOfYearNo,
       EXTRACT(YEAR_MONTH FROM OrderDate) AS YearMonth
FROM Orders
LIMIT 3;
```

RESULT

OrderDate	MonthEnd	DayOfWeekNo	DayOfYearNo	YearMonth
2026-01-05	2026-01-31	2	5	202601
2026-01-12	2026-01-31	2	12	202601
2026-01-20	2026-01-31	3	20	202601

3 rows in set

Problem 4.3.6 STR_TO_DATE: convert Indian-style text '15-08-2026' into a real DATE.

SOLUTION (SQL)

```
SELECT STR_TO_DATE('15-08-2026', '%d-%m-%Y') AS IndependenceDay2026,
       DAYNAME(STR_TO_DATE('15-08-2026', '%d-%m-%Y')) AS FallsOn;
```

RESULT

IndependenceDay2026	FallsOn
2026-08-15	Saturday

1 row in set

Problem 4.3.7 TIME functions: TIMEDIFF / ADDTIME / TIME_TO_SEC / SEC_TO_TIME / MAKEDATE.

SOLUTION (SQL)

```
SELECT TIMEDIFF('18:30:00', '09:15:00') AS ShiftLength,
       ADDTIME('09:15:00', '01:30:00') AS AfterBreak,
       TIME_TO_SEC('01:00:00') AS SecondsInHour,
       SEC_TO_TIME(5400) AS Seconds5400,
       MAKEDATE(2026, 100) AS Day100of2026;
```

RESULT

ShiftLength	AfterBreak	SecondsInHour	Seconds5400	Day100of2026
09:15:00	10:45:00	3600	01:30:00	2026-04-10

1 row in set

4.4 Advanced Functions

Control flow, conversion and system information functions.

Problem 4.4.1 IF(): label each order line as Big or Small.

SOLUTION (SQL)

```
SELECT OrderID, ProductID, Quantity,
       IF(Quantity >= 25, 'Big', 'Small') AS OrderSize
FROM OrderDetails
WHERE OrderID IN (1001, 1008);
```

RESULT

OrderID	ProductID	Quantity	OrderSize
1001	1	10	Small
1001	5	20	Small
1001	6	15	Small
1008	8	40	Big
1008	3	60	Big

5 rows in set

Tip: SQL Server / MS Access use IIF(condition, true_value, false_value).

Problem 4.4.2 CAST / CONVERT: change data types.

SOLUTION (SQL)

```
SELECT CAST('2026-03-15' AS DATE)      AS TextToDate,
       CAST(399.99 AS SIGNED)           AS DecimalToInt,
       CONVERT(250, CHAR)              AS NumberToText,
       CAST(Price AS CHAR) AS PriceText
FROM Products
LIMIT 1;
```

RESULT

TextToDate	DecimalToInt	NumberToText	PriceText
2026-03-15	400	250	250.00

1 row in set

Problem 4.4.3 COALESCE / IFNULL / NULLIF / ISNULL together.

SOLUTION (SQL)

```
SELECT CustomerName,
       IFNULL(PostalCode, 'none')      AS IfNullVal,
       COALESCE(PostalCode, City)      AS CoalesceVal,
       ISNULL(PostalCode)              AS IsItNull,
       NULLIF(Country, 'UAE')         AS NullIfUAE
FROM Customers
WHERE Country IN ('UAE', 'UK');
```

RESULT

CustomerName	IfNullVal	CoalesceVal	IsItNull	NullIfUAE
London Spice House	E1 6QL	E1 6QL	0	UK
Curry Corner	M1 5AN	M1 5AN	0	UK
Dubai Gourmet	none	Dubai	1	NULL
Sharjah Traders	none	Sharjah	1	NULL

4 rows in set

Problem 4.4.4 System information: current database, user and server version.

SOLUTION (SQL)

```
SELECT DATABASE() AS CurrentDB, CURRENT_USER() AS CurrentUser, VERSION() AS ServerVersion;
```

RESULT

```
+-----+-----+-----+
| CurrentDB | CurrentUser | ServerVersion |
+-----+-----+-----+
| ShopDB    | root@localhost | 10.11.14-MariaDB-0ubuntu0.24.04.1 |
+-----+-----+-----+
1 row in set
```

Tip: Values depend on your installation.

Problem 4.4.5 BIN / CONV: number system conversions.

SOLUTION (SQL)

```
SELECT BIN(10) AS Binary10, CONV('ff', 16, 10) AS HexFFtoDecimal, CONV(255, 10, 2) AS DecToBinary;
```

RESULT

```
+-----+-----+-----+
| Binary10 | HexFFtoDecimal | DecToBinary |
+-----+-----+-----+
| 1010     | 255             | 11111111    |
+-----+-----+-----+
1 row in set
```

4.5 Same task, different database - translation table

W3Schools documents three function sets (MySQL, SQL Server, MS Access). Use this table to translate any solution in this book.

Task	MySQL	SQL Server	MS Access	Oracle
First N rows	LIMIT n	TOP n	TOP n	FETCH FIRST n ROWS ONLY / ROWNUM
Join text	CONCAT(a, b)	a + b or CONCAT	a & b	a b
Text length	LENGTH / CHAR_LENGTH	LEN	Len	LENGTH
Part of text	SUBSTRING / MID	SUBSTRING	Mid	SUBSTR
Find in text	INSTR / LOCATE	CHARINDEX	InStr	INSTR
Upper case	UPPER / UCASE	UPPER	UCase	UPPER
Replace NULL	IFNULL / COALESCE	ISNULL / COALESCE	IIF(IsNull(x), ..)	NVL / COALESCE
If-then-else	IF(c, a, b) / CASE	IIF(c, a, b) / CASE	IIF(c, a, b)	CASE / DECODE
Current date+time	NOW()	GETDATE()	Now()	SYSDATE
Add to a date	DATE_ADD(d, INTERVAL 7 DAY)	DATEADD(day, 7, d)	DateAdd('d', 7, d)	d + 7
Date difference	DATEDIFF(d1, d2)	DATEDIFF(day, d2, d1)	DateDiff('d', d2, d1)	d1 - d2
Format a date	DATE_FORMAT(d, '%d-%m-%Y')	FORMAT(d, 'dd-MM-yyyy')	Format(d, 'dd-mm-yyyy')	TO_CHAR(d, 'DD-MM-YYYY')
Auto number	AUTO_INCREMENT	IDENTITY(1,1)	AUTOINCREMENT	GENERATED AS IDENTITY
Copy into new table	CREATE TABLE t AS SELECT	SELECT * INTO t FROM	SELECT * INTO t FROM	CREATE TABLE t AS SELECT
Run a procedure	CALL p(args)	EXEC p @x = 1	-	EXEC p(args)
Change column type	MODIFY COLUMN	ALTER COLUMN	ALTER COLUMN	MODIFY

Practice Challenges

Real business questions - try them without looking, then check the solutions

5.1 The challenge list (try these first!)

Everything you need was covered in Chapters 2-4. Write each query yourself before turning the page.

#	Challenge	Skills
1	Who are our top 3 customers by total amount spent?	JOIN x3, SUM, GROUP BY, LIMIT
2	Show the revenue for each month of 2026.	Date functions, GROUP BY
3	Which product sold the most units?	SUM, ORDER BY, LIMIT
4	Which products have never been ordered?	NOT IN subquery
5	Employee performance: orders handled and revenue generated by each employee (include employees with zero).	LEFT JOIN, COUNT DISTINCT, IFNULL
6	Which customers bought Saffron?	JOIN, DISTINCT, WHERE
7	What is the average order value?	Derived table, AVG
8	Which customers ordered more than once?	GROUP BY, HAVING
9	What is the second highest product price?	Subquery, MAX
10	Which orders contain more than 2 different products?	GROUP BY, HAVING
11	Revenue by customer country, split into domestic (India) and international.	CASE, GROUP BY
12	Which shipper carries the most international orders?	JOIN, WHERE, GROUP BY
13	For each category, show the most expensive product (correlated subquery).	Correlated subquery
14	Find suppliers whose products have generated more than 30,000 in sales.	JOIN, HAVING
15	Bonus (window function, MySQL 8+): rank customers by spend within their country.	Window function RANK()

5.2 Solutions

Challenge 1 Who are our top 3 customers by total amount spent?

SOLUTION (SQL)

```
SELECT c.CustomerName, SUM(od.Quantity * p.Price) AS TotalSpent
FROM Customers c
JOIN Orders o      ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID    = od.OrderID
JOIN Products p    ON od.ProductID = p.ProductID
GROUP BY c.CustomerID, c.CustomerName
ORDER BY TotalSpent DESC
LIMIT 3;
```

RESULT

```
+-----+-----+
| CustomerName | TotalSpent |
+-----+-----+
| Dubai Gourmet | 38400.00 |
| Sharjah Traders | 28260.00 |
| New York Masala | 19111.20 |
+-----+-----+
3 rows in set
```

Challenge 2 Show the revenue for each month of 2026.

SOLUTION (SQL)

```
SELECT MONTHNAME(o.OrderDate) AS Month, SUM(od.Quantity * p.Price) AS Revenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p    ON od.ProductID = p.ProductID
WHERE YEAR(o.OrderDate) = 2026
GROUP BY MONTH(o.OrderDate), MONTHNAME(o.OrderDate)
ORDER BY MONTH(o.OrderDate);
```

RESULT

```
+-----+-----+
| Month | Revenue |
+-----+-----+
| January | 28369.00 |
| February | 20110.00 |
| March | 54114.50 |
| April | 22311.20 |
| May | 15512.50 |
| June | 36260.00 |
+-----+-----+
6 rows in set
```

Challenge 3 Which product sold the most units?

SOLUTION (SQL)

```
SELECT p.ProductName, SUM(od.Quantity) AS UnitsSold
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName
ORDER BY UnitsSold DESC
LIMIT 1;
```

RESULT

```
+-----+-----+
| ProductName | UnitsSold |
+-----+-----+
| Mango Juice | 132 |
+-----+-----+
1 row in set
```

Challenge 4 Which products have never been ordered?

SOLUTION (SQL)

```
SELECT ProductName
FROM Products
WHERE ProductID NOT IN (SELECT DISTINCT ProductID FROM OrderDetails);
```

RESULT

```
+-----+
| ProductName |
+-----+
| Gulab Jamun Mix |
+-----+
1 row in set
```

Challenge 5 Employee performance: orders handled and revenue generated by each employee (include employees with zero).

SOLUTION (SQL)

```
SELECT CONCAT(e.FirstName, ' ', e.LastName) AS Employee,
       COUNT(DISTINCT o.OrderID) AS Orders,
       IFNULL(SUM(od.Quantity * p.Price), 0) AS Revenue
FROM Employees e
LEFT JOIN Orders o ON e.EmployeeID = o.EmployeeID
LEFT JOIN OrderDetails od ON o.OrderID = od.OrderID
LEFT JOIN Products p ON od.ProductID = p.ProductID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
ORDER BY Revenue DESC;
```

RESULT

```
+-----+-----+-----+
| Employee | Orders | Revenue |
+-----+-----+-----+
| Sara Thomas | 5 | 90092.20 |
| Vikram Singh | 3 | 45980.00 |
| Rohan Desai | 4 | 25025.00 |
| Kavya Iyer | 2 | 9740.00 |
| Arjun Mehta | 1 | 5840.00 |
| Neha Joshi | 0 | 0.00 |
+-----+-----+-----+
6 rows in set
```

Challenge 6 Which customers bought Saffron?

SOLUTION (SQL)

```
SELECT DISTINCT c.CustomerName, c.Country
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE p.ProductName = 'Saffron';
```

RESULT

```
+-----+-----+
| CustomerName | Country |
+-----+-----+
| London Spice House | UK |
| Berlin Bazaar | Germany |
| New York Masala | USA |
+-----+-----+
3 rows in set
```

Challenge 7 What is the average order value?

SOLUTION (SQL)

```
SELECT ROUND(AVG(OrderTotal), 2) AS AvgOrderValue
FROM (
  SELECT od.OrderID, SUM(od.Quantity * p.Price) AS OrderTotal
  FROM OrderDetails od
  JOIN Products p ON od.ProductID = p.ProductID
  GROUP BY od.OrderID
) AS t;
```

RESULT

```
+-----+
| AvgOrderValue |
+-----+
|      11778.48 |
+-----+
1 row in set
```

Challenge 8 Which customers ordered more than once?

SOLUTION (SQL)

```
SELECT c.CustomerName, COUNT(o.OrderID) AS Orders
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.CustomerName
HAVING COUNT(o.OrderID) > 1;
```

RESULT

```
+-----+-----+
| CustomerName | Orders |
+-----+-----+
| Sharma General Store |      2 |
| Patel Supermart      |      2 |
| Desi Mart            |      2 |
+-----+-----+
3 rows in set
```

Challenge 9 What is the second highest product price?

SOLUTION (SQL)

```
SELECT MAX(Price) AS SecondHighestPrice
FROM Products
WHERE Price < (SELECT MAX(Price) FROM Products);
```

RESULT

```
+-----+
| SecondHighestPrice |
+-----+
|          650.00 |
+-----+
1 row in set
```

Challenge 10 Which orders contain more than 2 different products?

SOLUTION (SQL)

```
SELECT OrderID, COUNT(*) AS LineItems
FROM OrderDetails
GROUP BY OrderID
HAVING COUNT(*) > 2;
```

RESULT

```
+-----+-----+
| OrderID | LineItems |
+-----+-----+
| 1001 | 3 |
| 1003 | 3 |
| 1005 | 3 |
| 1009 | 3 |
| 1010 | 3 |
| 1014 | 3 |
+-----+-----+
6 rows in set
```

Challenge 11 Revenue by customer country, split into domestic (India) and international.

SOLUTION (SQL)

```
SELECT c.Country,
       CASE WHEN c.Country = 'India' THEN 'Domestic' ELSE 'International' END AS Market,
       SUM(od.Quantity * p.Price) AS Revenue
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY c.Country
ORDER BY Revenue DESC;
```

RESULT

```
+-----+-----+-----+
| Country | Market | Revenue |
+-----+-----+-----+
| UAE | International | 66660.00 |
| India | Domestic | 52485.00 |
| UK | International | 24986.50 |
| USA | International | 19111.20 |
| Germany | International | 13434.50 |
+-----+-----+-----+
5 rows in set
```

Challenge 12 Which shipper carries the most international orders?

SOLUTION (SQL)

```
SELECT s.ShipperName, COUNT(*) AS InternationalOrders
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Shippers s ON o.ShipperID = s.ShipperID
WHERE c.Country <> 'India'
GROUP BY s.ShipperName
ORDER BY InternationalOrders DESC;
```

RESULT

```
+-----+-----+
| ShipperName | InternationalOrders |
+-----+-----+
| DHL Global | 6 |
| Delhivery | 1 |
+-----+-----+
2 rows in set
```

Challenge 13 For each category, show the most expensive product (correlated subquery).

SOLUTION (SQL)

```
SELECT c.CategoryName, p.ProductName, p.Price
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE p.Price = (SELECT MAX(p2.Price) FROM Products p2
                WHERE p2.CategoryID = p.CategoryID)
ORDER BY p.Price DESC;
```

RESULT

```
+-----+-----+-----+
| CategoryName | ProductName | Price |
+-----+-----+-----+
| Grains       | Basmati Rice | 780.00 |
| Dairy        | Ghee         | 650.00 |
| Sweets       | Kaju Katli   | 540.00 |
| Spices       | Saffron      | 438.90 |
| Beverages    | Filter Coffee | 320.00 |
| Snacks       | Bhujia Namkeen | 110.00 |
+-----+-----+-----+
6 rows in set
```

Challenge 14 Find suppliers whose products have generated more than 30,000 in sales.

SOLUTION (SQL)

```
SELECT s.SupplierName, SUM(od.Quantity * p.Price) AS Sales
FROM Suppliers s
JOIN Products p ON s.SupplierID = p.SupplierID
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY s.SupplierID, s.SupplierName
HAVING Sales > 30000
ORDER BY Sales DESC;
```

RESULT

```
+-----+-----+
| SupplierName | Sales |
+-----+-----+
| Gulf Fresh LLC | 71220.00 |
| Himalaya Foods | 38040.00 |
+-----+-----+
2 rows in set
```

Challenge 15 Bonus (window function, MySQL 8+): rank customers by spend within their country.

SOLUTION (SQL)

```
SELECT Country, CustomerName, TotalSpent,
       RANK() OVER (PARTITION BY Country ORDER BY TotalSpent DESC) AS RankInCountry
FROM (
  SELECT c.Country, c.CustomerName, SUM(od.Quantity * p.Price) AS TotalSpent
  FROM Customers c
  JOIN Orders o      ON c.CustomerID = o.CustomerID
  JOIN OrderDetails od ON o.OrderID   = od.OrderID
  JOIN Products p    ON od.ProductID = p.ProductID
  GROUP BY c.Country, c.CustomerName
) AS t
ORDER BY Country, RankInCountry;
```

RESULT

Country	CustomerName	TotalSpent	RankInCountry
Germany	Berlin Bazaar	7594.50	1
Germany	Munich Deli	5840.00	2
India	Desi Mart	17720.00	1
India	Patel Supermart	10130.00	2
India	Sharma General Store	9975.00	3
India	Mehta Kirana	8120.00	4
India	Fresh Basket	6540.00	5
UAE	Dubai Gourmet	38400.00	1
UAE	Sharjah Traders	28260.00	2
UK	London Spice House	15314.00	1
UK	Curry Corner	9672.50	2
USA	New York Masala	19111.20	1

12 rows in set

Appendix - SQL Keyword Index & Quick Reference

Every keyword from the W3Schools SQL Keywords reference, mapped to where you practise it

A.1 Keyword index

Keyword	Section	Keyword	Section
ADD	3.6	FOREIGN KEY	3.11
ADD CONSTRAINT	3.9, 3.11	FROM	2.1
ALL	2.35	FULL OUTER JOIN	2.27
ALTER	3.6	GROUP BY	2.31
ALTER COLUMN	3.6, 3.13	HAVING	2.32
ALTER TABLE	3.6	IN	2.8
ALTER VIEW	3.17 (CREATE OR REPLACE)	INDEX	3.14
AND	2.5	INNER JOIN	2.24
ANY	2.34	INSERT INTO	2.8
AS	2.22	INSERT INTO SELECT	2.37
ASC	2.4	IS NULL	2.9
BACKUP DATABASE	3.3	IS NOT NULL	2.9
BETWEEN	2.21	JOIN	2.23
CASE	2.38	LEFT JOIN	2.25
CHECK	3.12	LIKE	2.18
COLUMN	3.6	LIMIT	2.12
CONSTRAINT	3.7	NOT	2.7
CREATE	3.1, 3.4	NOT NULL	3.8
CREATE DATABASE	1.1, 3.1	OR	2.4
CREATE INDEX	3.14	ORDER BY	2.4
CREATE OR REPLACE VIEW	3.17	OUTER JOIN	2.25, 2.26, 2.27
CREATE TABLE	1.1, 3.4	PRIMARY KEY	3.10
CREATE PROCEDURE	2.40	PROCEDURE	2.40
CREATE UNIQUE INDEX	3.14	RIGHT JOIN	2.26
CREATE VIEW	3.17	ROWNUM	2.12 (Oracle)
DATABASE	3.1, 3.2	SELECT	2.1
DEFAULT	3.13	SELECT DISTINCT	2.2
DELETE	2.11	SELECT INTO	2.36
DESC	2.4	SELECT TOP	2.12
DISTINCT	2.2	SET	2.10
DROP	3.2, 3.5	TABLE	3.4
DROP COLUMN	3.6	TOP	2.12
DROP CONSTRAINT	3.12	TRUNCATE TABLE	3.5
DROP DATABASE	3.2	UNION	2.29

Keyword	Section	Keyword	Section
DROP DEFAULT	3.13	UNION ALL	2.30
DROP INDEX	3.9, 3.14	UNIQUE	3.9
DROP TABLE	3.5	UPDATE	2.10
DROP VIEW	3.17	VALUES	2.8
EXEC	2.40 (SQL Server)	VIEW	3.17
EXISTS	2.33	WHERE	2.3

A.2 One-page quick reference

Task	Pattern
SELECT	<code>SELECT col1, col2 FROM t WHERE cond ORDER BY col1 DESC LIMIT 10;</code>
Aggregate + group	<code>SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1;</code>
Join	<code>SELECT a.x, b.y FROM a INNER LEFT RIGHT JOIN b ON a.id = b.a_id;</code>
Insert	<code>INSERT INTO t (c1, c2) VALUES (v1, v2), (v3, v4);</code>
Update	<code>UPDATE t SET c1 = v1 WHERE cond;</code>
Delete	<code>DELETE FROM t WHERE cond;</code>
Subquery	<code>SELECT * FROM t WHERE c IN (SELECT c FROM t2 WHERE cond);</code>
Create table	<code>CREATE TABLE t (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50) NOT NULL);</code>
Alter table	<code>ALTER TABLE t ADD c INT; ALTER TABLE t DROP COLUMN c;</code>
Index / view	<code>CREATE INDEX ix ON t (c); CREATE VIEW v AS SELECT ...;</code>
Procedure	<code>CREATE PROCEDURE p(IN x INT) BEGIN ... END; CALL p(1);</code>
Prepared	<code>PREPARE s FROM 'SELECT ... WHERE id = ?'; EXECUTE s USING @id;</code>

Logical order of execution (why aliases from SELECT cannot be used in WHERE): FROM / JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT.